

РЕФЕРАТ

Расчетно-пояснительная записка к выпускной квалификационной работе «Метод обнаружения фальсифицированного фрагмента на изображении с использованием сверточной нейронной сети» содержит 67 страниц, 4 части, 19 рисунков, 4 таблицы, список используемых источников из 34 наименований и 2 приложения.

Ключевые слова: компьютерное зрение, глубокое обучение, сверточная нейронная сеть.

Объект разработки – метод обнаружения фальсифицированного фрагмента на изображении.

Цель работы: разработка и программная реализация метода автоматизированного определения области фальсифицированного фрагмента изображения с помощью двухпоточной свёрточной нейронной сети.

В первой части работы приведены обзор и сравнение существующих методов обнаружения фальсифицированного фрагмента на изображении.

Во второй части описан метод обнаружения фальсифицированного фрагмента на изображении с использованием сверточной нейронной сети, приведены IDEF0-диаграммы метода и схемы его алгоритмов.

В третьей части обоснован выбор средств разработки, приведена структура ПО, реализующий предложенный метод и описано взаимодействие пользователя с программным обеспечением..

В четвертой части выбраны метрики для оценки качества работы метода и приведены результаты исследования их зависимости от конфигурации нейронной сети и объемов передаваемых данных.

Разработанный метод может найти применение в области информационной безопасности и верификации медиаконтента в средствах массовой информации и социальных сетях. Это позволит своевременно выявлять и блокировать распространение дезинформации.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ВВЕДЕНИЕ	8
1 Аналитический раздел	9
1.1. Фальсификация изображений и задача её обнаружения	9
1.1.1. Понятие фальсификации изображений	9
1.1.2. Основные трудности и требования к методам обнаружения	10
1.2. Методы обнаружения фальсифицированного фрагмента на изображении	12
1.2.1. Методы на основе анализа целостности изображений	12
1.2.2. Методы поиска ключевых точек	16
1.2.3. Метод на основе сверточной нейронной сети	19
1.3. Сравнительный анализ методов фальсифицированного фрагмента на изображении	21
1.3.1. Обоснование критериев сравнения методов	21
1.3.2. Таблица сравнения методов	22
1.4. Формализованная постановка задачи	23
2 Конструкторский раздел	25
2.1. Требования и ограничения к разрабатываемому методу	25
2.2. Описание разрабатываемого метода	25
2.2.1. Декомпозиция разрабатываемого метода	25
2.2.2. Алгоритмы предобработки и извлечения признаков	26
2.2.3. Структура разрабатываемой сверточной нейронной сети	29
2.3. Структура разрабатываемого программного обеспечения	34
2.4. Набор данных для обучения модели	35
3 Технологический раздел	37
3.1. Выбор средств реализации программного обеспечения	37
3.1.1. Выбор языка программирования	37
3.1.2. Выбор среды программирования	37
3.2. Формат входных и выходных данных	39

3.3.	Структура разрабатываемого программного обеспечения	40
3.4.	Руководство пользователя	40
3.5.	Интерфейс пользователя	42
4	Исследовательский раздел	45
4.1.	Метрики оценки качества модели	45
4.2.	Технические характеристики	46
4.3.	Влияние слоя внимания на качество модели	47
4.4.	Влияние слоя адаптации признаков на качество модели .	48
4.5.	Влияние объема передаваемых данных на качество работы метода	50
4.6.	Влияние качества изображения на качество модели . . .	51
	ЗАКЛЮЧЕНИЕ	54
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	55
	ПРИЛОЖЕНИЕ А	59
	ПРИЛОЖЕНИЕ Б	67

ВВЕДЕНИЕ

В условиях быстрого развития технологий в последние несколько лет поддельные мультимедиа стали центральной проблемой. Используя передовые инструменты, создание поддельных медиапродуктов, таких как изображения и видео, становится очень простым, если доступны большие объёмы данных. Создание поддельного медиаконтента неэтично и представляет угрозу для общества. Они широко используются в киберпреступности для кражи личных данных, кибервымогательства, фейковых новостей, финансового мошенничества и многого другого. Поэтому больше внимания следует уделять выявлению контрафактной продукции. Обнаружение поддельного контента является серьёзной проблемой и пользуется большим спросом в сфере цифровой безопасности.

Особую сложность среди всех видов поддельных медиа представляют фальсифицированные изображения, а именно те случаи, когда изменён не весь кадр, а лишь отдельный фрагмент. Такие локальные манипуляции сложнее обнаружить, так как большая часть изображения остаётся подлинной. Современные генеративные модели способны создавать высокореалистичные фрагменты, визуально неотличимые от оригинала, что требует разработки новых, более совершенных алгоритмов обнаружения фальсификации.

Цель работы — разработка и программная реализация метода автоматизированного определения области фальсифицированного фрагмента изображения с помощью двухпоточной свёрточной нейронной сети.

Для достижения поставленной цели необходимо выполнить следующие задачи:

- провести обзор и сравнение существующих методов обнаружения фальсифицированного фрагмента на изображении;
- разработать метод обнаружения фальсифицированного фрагмента на изображении с использованием свёрточной нейронной сети, привести IDEF0-диаграммы метода и схемы его алгоритмов;
- разработать программное обеспечение, реализующее данный метод;
- выбрать метрики для оценки качества работы метода и провести исследования качества работы метода.

1 Аналитический раздел

1.1. Фальсификация изображений и задача её обнаружения

1.1.1. Понятие фальсификации изображений

Под фальсификацией (подделкой) цифровых изображений понимается процесс внесения преднамеренных изменений в исходное изображение с целью искажения визуального содержания или смысла, который оно передаёт. В отличие от обычной обработки (коррекция яркости, цветокоррекция, шумоподавление), фальсификация направлена на создание ложного представления о реальности, что может использоваться как в личных, так и в профессиональных или политических целях [1].

С развитием доступных графических редакторов (Adobe Photoshop, GIMP и др.) и методов глубокого обучения возможности по созданию реалистичных подделок значительно расширились. В связи с этим задача обнаружения фальсификаций становится особенно актуальной в областях журналистики, криминалистики и социальных медиа[2].

В современной литературе по компьютерному зрению принято выделять несколько основных типов фальсификации изображений[3].

- Копирование-вставка (англ. copy-move) — фрагмент изображения копируется и вставляется в другую область того же изображения, часто для сокрытия объектов или их дублирования.
- Вставка объектов из другого изображения (англ. splicing) — часть одного изображения переносится в другое, создавая сцену, которой не существовало в действительности.
- Ретуширование (англ. inpainting) — удаление или замена элементов с помощью заполнения области (удаление теней, людей, артефактов, ...).
- Генерация полностью синтетических изображений — создание изображений с нуля с использованием генеративно-состязательных сетей (GAN) или диффузионных моделей.
- Deepfake — замена лица или мимики человека на видео и изображениях с использованием нейросетевых методов.

Ниже, на рисунке 1.1, представлены примеры фальсификаций изображений типа копирование-вставка.

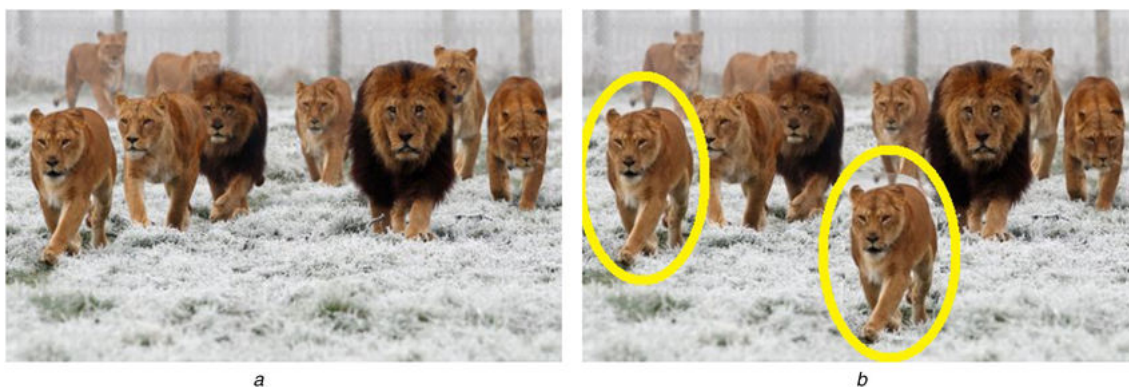


Рисунок 1.1 – Пример фальсификаций изображений типа копирование-вставка.

Цифровая фальсификация оставляет определённые статистические и семантические следы. Например, при копировании-вставке возникает повторяющаяся структура блоков, при вставке — нарушение согласованности шума датчика или освещения, а при ретушировании — аномалии в частотной области (артефакты сглаживания, некорректное сохранение JPEG)[4].

Важно отметить, что понятие фальсификации не включает в себя такие изменения, как сжатие с потерями, масштабирование или поворот, если они не направлены на введение в заблуждение зрителя. Однако в реальных условиях поддельное изображение часто подвергается дополнительным операциям (JPEG-сжатие, изменение размера, фильтрация), что усложняет его обнаружение[5].

Таким образом, для обнаружения фальсифицированного фрагмента необходимо учитывать множество потенциальных аномалий на различных уровнях представления изображения: от пиксельного до семантического.

1.1.2. Основные трудности и требования к методам обнаружения

Задача обнаружения фальсифицированного фрагмента на изображении относится к классу задач пассивной судебной экспертизы изображений. В отличие от активных методов (цифровая подпись, водяные знаки), пассивные методы не требуют предварительного внедрения вспомогательной информации, однако сталкиваются с рядом принципиальных трудностей, обусловленных как

свойствами самих изображений, так и действиями злоумышленника[6].

К числу основных трудностей относятся.

1. Отсутствие эталонного («чистого») изображения. В большинстве практических сценариев (например, при проверке фотографий в СМИ или на интернет-платформах) исходное неизменённое изображение недоступно. Поэтому метод должен выявлять аномалии исключительно на основе статистических и семантических характеристик самого анализируемого файла.
2. Разнообразие типов фальсификаций. Как отмечалось в п. 1.1.1, подделки могут быть выполнены методами copy-move, splicing, inpainting, deepfake и др. Каждый тип оставляет различные следы: повторяющиеся блоки, несоответствие шума сенсора, артефакты интерполяции или генеративные контуры. Универсальный метод должен быть устойчивым к нескольким типам подделок [7].
3. Соккрытие следов подделки послеоперационной обработкой. Злоумышленник может применять к поддельному изображению сжатие JPEG, масштабирование, добавление шума, медианную фильтрацию или размытие. Эти манипуляции намеренно затрудняют обнаружение артефактов, разрушая статистические аномалии, внесённые в момент фальсификации.
4. Разнородность исходных изображений. Метод не должен полагаться на фиксированные характеристики (размер, разрешение, цветовое пространство, модель камеры), так как изображения в реальных условиях поступают из самых разных источников [8]. Это требует инвариантности к разрешению, коэффициенту сжатия и уровню шума.
5. Вычислительная сложность и требования к ресурсам и времени. Применение метода в потоковых системах (социальные сети, новостные агрегаторы) требует быстрой обработки (миллисекунды на изображение) и работы с большими объёмами данных. Слишком ресурсоёмкие модели неприменимы в реальном времени.

В связи с перечисленными трудностями к современным методам обнаружения фальсифицированного фрагмента предъявляются следующие требования [9]:

- Высокая точность: метод должен обеспечивать высокие значения метрик IoU, F1-меры и точности классификации пикселей на тестовых наборах, сбалансированных по типам подделок.
- Робастность (устойчивость): метод должен сохранять работоспособность после сжатия JPEG, изменения размера, добавления шума и других типов постобработки.
- Обобщающая способность: обученная модель должна корректно работать на изображениях, взятых из датасетов, не участвовавших в обучении, и на неизвестных типах подделок.
- Локализация фрагмента: метод должен не только выносить бинарное решение, но и выдавать карту локализации с указанием границ изменённой области (пиксельная маска).
- Инвариантность к изображению: метод не должен зависеть от конкретной модели камеры, формата (JPEG, PNG, BMP) и цветовой схемы (RGB, YCbCr).

Таким образом, разработка практического метода обнаружения фальсифицированного фрагмента представляет собой сложную многокритериальную задачу, требующую компромисса между точностью, устойчивостью и скоростью работы.

1.2. Методы обнаружения фальсифицированного фрагмента на изображении

1.2.1. Методы на основе анализа целостности изображений

Методы обнаружения фальсифицированного фрагмента на основе анализа целостности изображений опираются на предположение, что любая модификация исходного изображения нарушает определённые статистические закономерности, присущие естественным изображениям. В отличие от методов поиска ключевых точек, которые ориентированы преимущественно на выявление фальсификации типа копирование-вставка, подходы на основе целостности анализируют непротиворечивость таких физически обоснованных характеристик,

как шум датчика (PRNU), артефакты двойного сжатия JPEG и корреляции, вносимые интерполяцией цветных фильтров (CFA). Каждый из этих трёх методов имеет собственную область эффективности и ограничения. На практике эти методы часто применяются в комплексе: например, сначала выполняется анализ PRNU для грубой локализации подозрительных областей, а затем уточнение с помощью CFA-интерполяции. Такой многоэтапный подход позволяет снизить количество ложных срабатываний [2].

В работе [10] также показано, что эффективность PRNU-анализа резко падает, если фальсифицированный фрагмент был предварительно обесцвечен или подвергнут нелинейной гамма-коррекции. Для повышения робастности исследователи предлагают использовать вейвлет-преобразование Хаара, которое сохраняет высокочастотную составляющую шума даже при умеренном сжатии. Однако при коэффициенте качества JPEG ниже 75% PRNU-следы становятся практически неразличимыми. Кроме того, практическое применение метода требует, чтобы на изображении была как минимум одна область с заведомо неизменённым шумом. Такую область часто выделяют вручную или с помощью алгоритмов поиска однородных текстур. Без этого эталонного фрагмента корреляционный анализ теряет смысл. На практике PRNU-метод наиболее полезен при расследовании единичных целенаправленных фальсификаций, где камера-источник может быть изъята и исследована. В потоковой обработке интернет-изображений он практически не применяется из-за отсутствия эталонных шумов.

Метод на основе анализа шума датчика (PRNU)

Каждая цифровая камера оставляет уникальный «отпечаток» — фотоотклик неравномерности (photo-response non-uniformity, PRNU), возникающий из-за неоднородной чувствительности пикселей [4]. При вставке фрагмента из другого изображения (splicing) в подделанной области нарушается согласованность PRNU: появляется шум другой камеры или его отсутствие. Метод вычисляет PRNU анализируемого изображения (обычно с помощью вейвлет-фильтрации или использования фильтра высоких частот) и оценивает его корреляцию с эталонным шумом предполагаемой камеры. Если фрагмент изображения имеет статистически значимое расхождение в структуре шума, это свидетельствует о фальсификации.

Ниже, на рисунке 1.2, представлена схема работы метода на основе ана-

лиза шума датчика (PRNU).

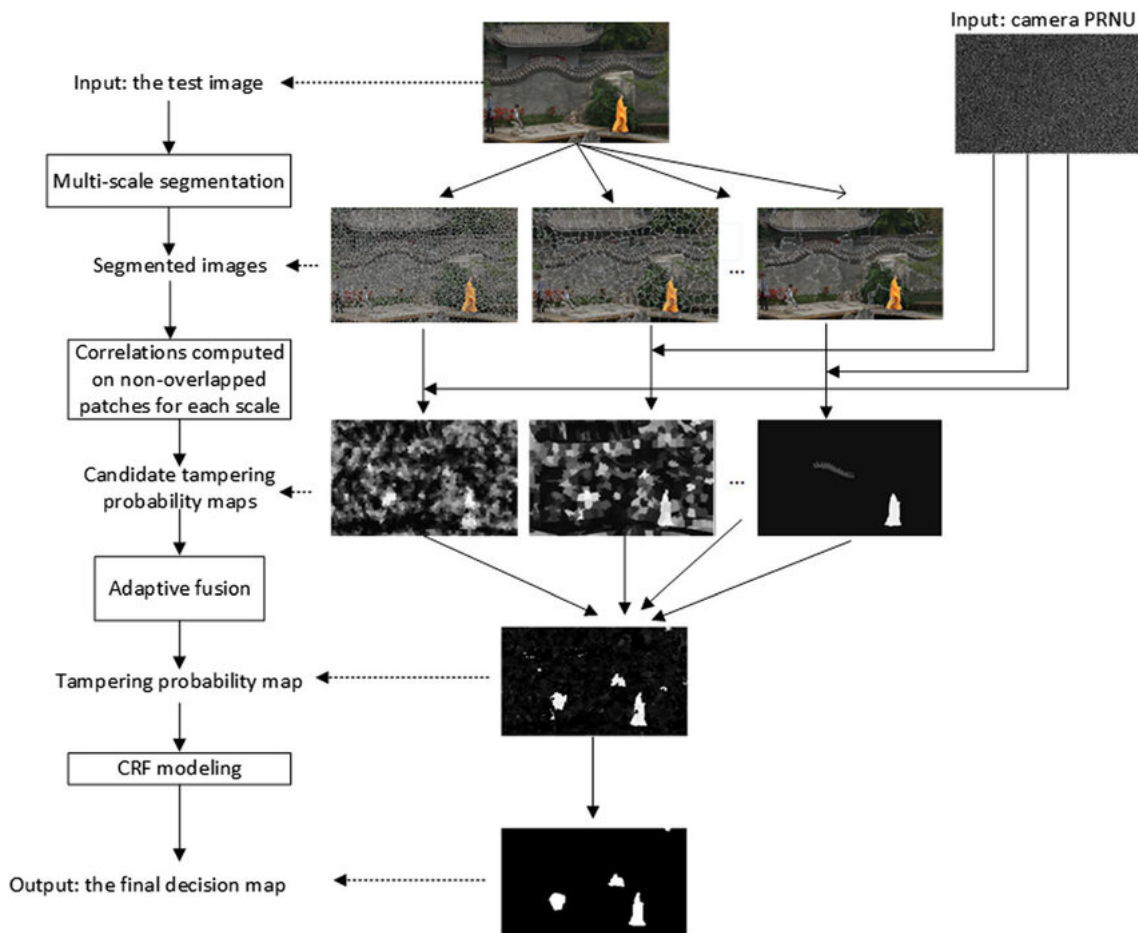


Рисунок 1.2 – Схема работы метода на основе анализа шума датчика.

Метод на основе двойного сжатия JPEG (Double JPEG)

Большинство цифровых изображений сохраняются в формате JPEG с потерями. Если злоумышленник редактирует изображение и сохраняет его повторно, возникает так называемое двойное сжатие с возможно разными таблицами квантования. При этом гистограмма коэффициентов дискретного косинусного преобразования (DCT) для отдельных частотных полос приобретает характерные периодические пики. Метод анализирует гистограммы DCT-коэффициентов и выявляет наличие двух различных стадий квантования. В наиболее развитой форме метод не только определяет факт двойного сжатия, но и оценивает параметры первого сжатия [11]. Достоинства: работает без какой-либо априорной информации об исходной камере, устойчив к умеренным атакам. Недостатки: метод даёт в основном бинарный ответ и плохо локализует точную область вставки; кроме того, если злоумышленник сохранил подделку без сжатия (в формате PNG) или использовал тот же коэффициент качества, двойное сжатие может быть не обнаружено.

Важным уточнением является то, что метод двойного сжатия эффективен только для подделок, которые были сохранены в JPEG хотя бы один раз после редактирования. Если оригинал был в формате без потерь, а фальсификация тоже сохранена без потерь, то DCT-гистограммы не покажут аномалий. Для улучшения локализации применяется скользящее окно фиксированного размера. Окно последовательно перемещается по изображению, и для каждой позиции вычисляется вероятность двойного сжатия. Области, где эта вероятность превышает порог, объединяются в итоговую карту подозрительных фрагментов. Такой подход, хотя и вычислительно затратен, позволяет очертить границы вставки с точностью до размера окна. Кроме того, устойчивость метода можно повысить, анализируя не только гистограммы, но и корреляции между DCT-коэффициентами соседних блоков. В подделанных областях эти корреляции часто нарушаются из-за того, что один блок мог быть сжат с одними параметрами, а соседний — с другими. Данный приём, известный как «анализ блоковой когерентности», даёт хорошие результаты даже при повторном сжатии с тем же коэффициентом качества.

Метод на основе анализа интерполяции цветных фильтров (CFA)

Большинство цифровых камер используют матрицу цветных фильтров Байера, где каждый пиксель регистрирует только один цветовой канал (зелёный, красный или синий), а недостающие цветовые значения восстанавливаются с помощью демозаики. Процесс интерполяции вносит корреляции между соседними пикселями и между цветовыми каналами. При вставке фрагмента из другого изображения или при ретушировании локальная структура CFA-интерполяции нарушается, так как вставленная область имеет свою историю обработки. Метод моделирует ожидаемый процесс интерполяции для данной модели камеры или строит универсальную модель и вычисляет ошибку предсказания пикселя по соседним. В областях, где CFA-интерполяция нарушена, ошибка предсказания статистически значимо выше. Достоинства метода: возможность локализации подделки на уровне пикселей без необходимости знать эталонный PRNU. Недостатки: высокая чувствительность к сжатию JPEG, а также к масштабированию. Метод эффективен только для изображений, поступивших напрямую с камеры и сохранённых в формате без потерь или с минимальным сжатием [12].

На практике CFA-метод часто реализуют через обучение небольшой ней-

ронной сети предсказывать зелёный канал по красному и синему. Такую сеть обучают на заведомо подлинных изображениях, снятых различными камерами. При тестировании для каждого пикселя вычисляется ошибка предсказания; области с аномально высокой ошибкой маркируются как потенциально поддельные. Такой подход не требует явного задания модели интерполяции и частично компенсирует разницу между камерами разных производителей. Однако главное ограничение CFA-метода остаётся неизменным: он практически неработоспособен после сжатия JPEG с качеством ниже 90%. Даже небольшое сжатие разрушает тонкие корреляции, внесённые демозаикой. Более того, если поддельный фрагмент был предварительно масштабирован или повернут, то его собственная CFA-интерполяция уже не соответствует исходной сетке Байера, и метод может дать ложное срабатывание даже на подлинной области.

1.2.2. Методы поиска ключевых точек

Методы обнаружения фальсификации на основе поиска ключевых точек (keypoint-based methods) являются одним из наиболее эффективных подходов для выявления атак типа «копирование-вставка». Основная идея заключается в том, что при копировании фрагмента изображения и вставке его в другое место того же изображения в подделке появляются две или более области с идентичным или очень похожим содержанием. Ключевые точки представляют собой локальные особенности, инвариантные к масштабу, повороту и частично к изменению освещения, что позволяет обнаруживать дублированные фрагменты даже после их геометрического преобразования [13]. Наибольшее распространение в задачах криминалистики изображений получили SIFT (Scale-Invariant Feature Transform) и SURF (Speeded-Up Robust Features) [14].

Метод SIFT

Дескриптор SIFT, предложенный Лоу в 2004 году, стал золотым стандартом для обнаружения фальсификации типа копирование-вставка [15]. Алгоритм включает следующие этапы: построение масштабного пространства, локализацию экстремумов разности гауссия, отбраковку низкоконтрастных и граничных точек, назначение ориентации на основе градиентов и формирование 128-мерного дескриптора для каждой ключевой точки. При обнаружении подделки вычисляются SIFT-дескрипторы для всего изображения, затем выполняется их попарное сравнение (с использованием отношения расстояний до ближайшего

и второго ближайшего соседа). Пары ключевых точек с высокой степенью совпадения дескрипторов группируются в кластеры с помощью алгоритмов иерархической кластеризации или DBSCAN, после чего для каждой группы строится ограничивающий прямоугольник или точный контур подозрительной области [16]. SIFT демонстрирует высокую устойчивость к повороту, масштабированию и небольшому изменению перспективы, а также к сжатию JPEG при коэффициентах качества выше 70%. Однако его вычислительная сложность относительно высока: извлечение дескрипторов и особенно их попарное сравнение может занимать несколько секунд на изображении среднего разрешения [17].

Метод SURF

Для ускорения обработки был предложен дескриптор SURF, который использует интегральные изображения и приближённые детерминанты матрицы Гессе для быстрого обнаружения ключевых точек [18]. SURF формирует 64-мерный или 128-мерный дескриптор на основе распределения гауссовых вейвлет-ответов в окрестности точки.

Ниже, на рисунке 1.3, представлен пример ключевых точек SURF.

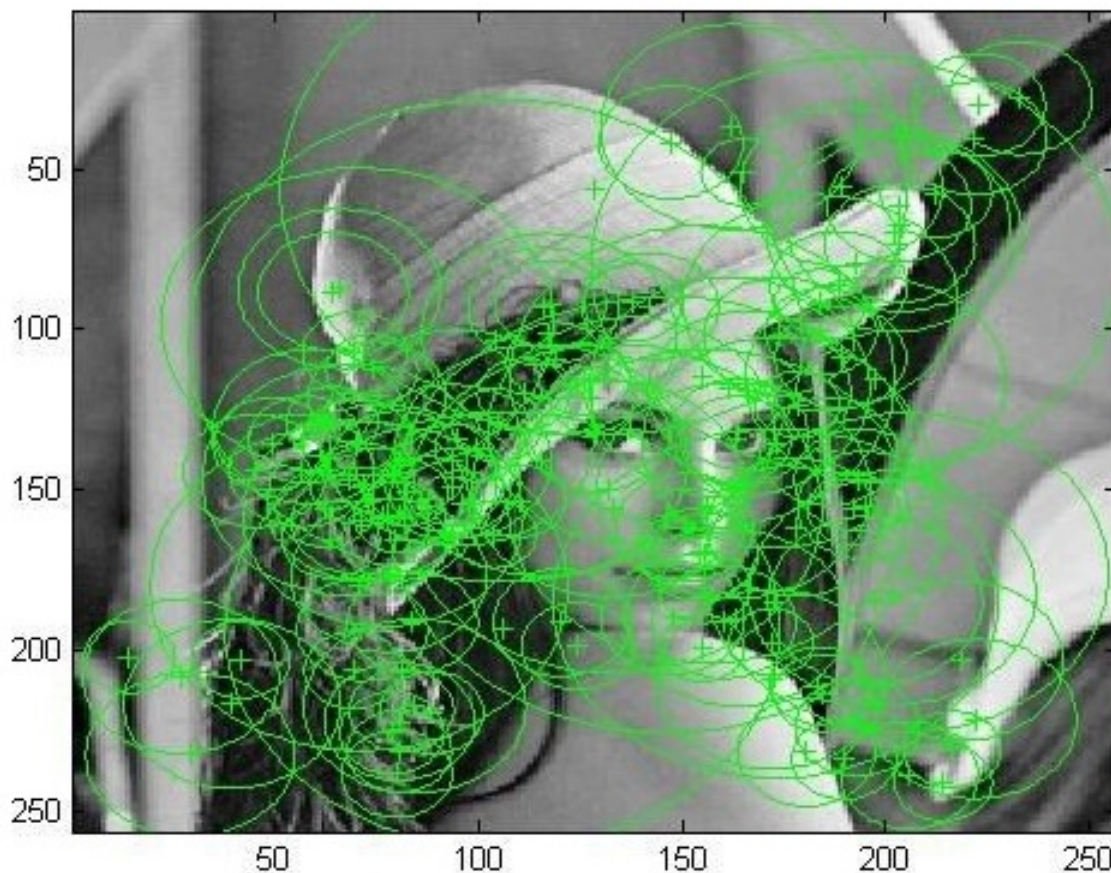


Рисунок 1.3 – Пример ключевых точек SURF.

По сравнению с SIFT, SURF работает в несколько раз быстрее при со-

поставимой точности для большинства типов фальсификаций копирование-вставка. Однако SURF уступает SIFT при сильных аффинных искажениях и при низкой освещённости. В задачах обнаружения фальсификаций SURF часто выбирают для систем реального времени или для обработки больших массивов изображений, когда время критичнее максимальной точности [19].

На практике разница в скорости становится заметной уже на изображениях размером 1024×768 пикселей: SURF обрабатывает такое изображение за 0.2–0.3 секунды, тогда как SIFT – за 1.5–2 секунды [18]. Для мобильных приложений или веб-сервисов, где требуется ответ менее чем за секунду, SURF оказывается предпочтительнее. Однако при работе с низкоконтрастными изображениями SURF генерирует значительно меньше устойчивых ключевых точек, чем SIFT, что может привести к пропуску фальсификации [20].

Важной особенностью SURF является возможность выбора размера дескриптора: 64-мерный вариант быстрее в сравнении, но даёт больше ложных совпадений на текстурированных фонах. 128-мерный вариант работает медленнее, но обеспечивает точность, близкую к SIFT. При реализации систем обнаружения копирование-вставка часто используют 64-мерный SURF на первом этапе грубого поиска, а затем верифицируют подозрительные кластеры с помощью более точного SIFT-дескриптора. Такой гибридный подход позволяет достичь компромисса между скоростью и точностью [19].

Кроме того, SURF, как и SIFT, не способен обнаружить фальсификации, если дублированная область была предварительно размыта или к ней применили нелинейную фильтрацию. Даже лёгкое медианное размытие с окном 3×3 снижает количество правильных совпадений на 60–70%. Поэтому при разработке системы, устойчивой к постобработке, дескрипторные методы рекомендуется дополнять фильтрацией высоких частот или анализом остаточных шумов.

Ограничения методов на основе ключевых точек

При применении SIFT/SURF для анализа больших коллекций изображений важно задать минимальное количество ключевых точек, необходимое для принятия решения о наличии копирование-вставки. Опыт показывает, что порог в 8–12 совпадающих точек достаточно надёжно отсеивает случайные совпадения, но может пропускать подделки с очень маленькими скопированными фрагментами [14]. Для обнаружения мелких вставок порог снижают до 4–5 точек, но при этом резко возрастает число ложных срабатываний на текстуриро-

ванных фонах. В таких случаях помогает дополнительная проверка формы области: если найденный кластер точек вытянут вдоль границы объекта, вероятность реальной подделки выше, чем для компактного кластера.

Ещё одной практической проблемой является то, что алгоритмы кластеризации требуют задания максимального евклидова расстояния между дескрипторами в пространстве признаков. Это расстояние зависит от степени сжатия изображения и уровня шума. Для JPEG с качеством 95% разумное пороговое значение составляет 0.6–0.7, а для качества 70% порог приходится увеличивать до 0.8–0.9, что ведёт к росту ложных совпадений. Поэтому перед анализом рекомендуется оценивать коэффициент качества JPEG и адаптивно подбирать порог. В автоматизированных системах это можно реализовать через предварительный анализ DCT-гистограммы.

Наконец, отметить, что SIFT и SURF дают наилучшие результаты при разрешении входного изображения не ниже 640×480 пикселей. При меньшем разрешении количество ключевых точек падает настолько, что даже очевидные сору-мове подделки могут остаться незамеченными. Для низкоразрешенных изображений более эффективными оказываются методы, основанные на блочном сравнении без ключевых точек, либо сверточные нейронные сети, работающие непосредственно с пикселями [21].

1.2.3. Метод на основе сверточной нейронной сети

Методы обнаружения фальсифицированных фрагментов на основе свёрточных нейронных сетей (CNN) в последние годы заняли доминирующее положение в области компьютерной криминалистики изображений. В отличие от классических подходов (анализ целостности или поиск ключевых точек), которые требуют ручного проектирования признаков, CNN автоматически извлекают иерархические представления из данных, обучаясь различать подлинные и поддельные области на больших размеченных наборах. Способность сетей выявлять сложные статистические аномалии, невидимые для традиционных детекторов, обусловила их широкое применение для выявления вставки объектов, копирования-вставки и так далее [22].

Первые работы по применению CNN для обнаружения подделок использовали простую архитектуру с несколькими свёрточными и полносвязными слоями, работавшую на входных изображениях фиксированного малого размера

(например, 64×64 или 128×128). Такие сети классифицировали небольшие патчи как подлинные или поддельные, после чего для локализации фальсификации применялся скользящий оконный метод [23]. Хотя этот подход демонстрировал приемлемую точность на сбалансированных датасетах, он был вычислительно затратным и давал высокую долю ложноположительных срабатываний на границах объектов. Кроме того, сети, обученные на патчах, часто запоминали артефакты конкретного типа фальсификации и плохо обобщались на другие типы фальсификаций.

Прорывом стало появление полностью свёрточных сетей (FCN — Fully Convolutional Networks), которые на входе принимают изображение произвольного размера и на выходе выдают карту локализации той же пространственной размерности. Архитектура типа U-Net (симметричный энкодер-декодер с пропускными соединениями) стала стандартом для пиксельной сегментации подделанных областей [24]. Энкодер последовательно уменьшает пространственное разрешение, извлекая всё более абстрактные признаки, а декодер восстанавливает детализацию и формирует маску подделки. Пропускные соединения позволяют сохранить мелкие гранулярные признаки, которые иначе теряются при многократном субдискретизации [25].

Важным направлением является использование не RGB-изображений в качестве входа, а так называемых остаточных карт, полученных вычитанием из исходного изображения его сглаженной версии. Такой подход подавляет семантическое содержание и усиливает следы камеры и постобработки. Сеть, обученная на остатках, оказывается более устойчивой к сжатию JPEG и изменениям контента, поскольку вынуждена фокусироваться на статистических аномалиях, а не на форме объектов [26].

Для борьбы с разнообразием типов подделок и усложняющих обработок исследователи разрабатывают многозадачные и многомасштабные архитектуры. Один из подходов — использование параллельных ветвей сети, работающих на разных масштабах, с последующим объединением карт признаков. Другой — применение механизмов внимания, которые позволяют сети динамически взвешивать важность различных пространственных областей и каналов признаков.

Одной из ключевых проблем CNN-методов является их чувствительность к различиям между обучающими и тестовыми датасетами. Сеть, обученная на изображениях с одними камерами и степенями сжатия, может резко терять точ-

ность при тестировании на других. Для повышения обобщающей способности применяют методы аугментации, где сеть обучается одновременно на подлинных и сгенерированных подделках, устойчивых к детектированию. В последнее время активно развиваются методы адаптации признаков, которые выравнивают распределения признаков из разных доменов с помощью градиентного реверса слоя или вариационных автоэнкодеров.

Несмотря на высокие показатели качества, CNN-методы имеют и недостатки. Во-первых, они требуют больших размеченных наборов данных с пиксельной маской подделки, создание которых трудоёмко. Во-вторых, многие модели «чёрный ящик»: сложно интерпретировать, на каком именно основании сеть отнесла область к подделанной. В-третьих, вычислительные затраты на обучение глубоких сетей высоки, и для работы в реальном времени часто требуется GPU-ускорение [2].

1.3. Сравнительный анализ методов фальсифицированного фрагмента на изображении

1.3.1. Обоснование критериев сравнения методов

Для объективного сопоставления различных методов обнаружения фальсифицированного фрагмента на изображении необходимо определить систему критериев, отражающих как точность детекции, так и пригодность метода для практического применения. В настоящей работе предлагается использовать три основных критериев: точность локализации фальсификации, устойчивость к постобработке и обобщающую способность на неизвестных типах подделок.

Первым и наиболее важным критерием является точность локализации подделанного фрагмента. В отличие от бинарной классификации, задача локализации требует указать границы изменённой области на пиксельном уровне. Высокое значение точности свидетельствует о том, что метод способен точно выделить форму вставленного объекта.

Второй критерий — устойчивость к постобработке изображения. В реальных условиях фальсифицированное изображение почти всегда подвергается сжатию с потерями, изменению размера или добавлению шума. Злоумышленник может намеренно применять такие операции для маскировки следов фальсификаций. Сравнение методов должно проводиться при разных коэффициен-

тах качества JPEG.

Третий критерий — обобщающая способность на неизвестных типах фальсификации и датасетах. Метод не должен «переобучаться» под конкретный датасет. Для оценки обобщения применяются обучение на одном наборе данных и тестирование на другом.

1.3.2. Таблица сравнения методов

На основе обоснованных в п. 1.3.1 критериев (точность локализации, робастность к постобработке, вычислительная эффективность, обобщающая способность, интерпретируемость) проведём сравнительный анализ трёх основных классов методов обнаружения фальсифицированного фрагмента: аналитических методов целостности, дескрипторных методов на основе ключевых точек и методов на основе свёрточных нейронных сетей (включая их расширения с вниманием и адаптацией признаков). Для каждого критерия используется качественная шкала: «высокая», «средняя», «низкая», а также уточняющие комментарии. Результаты сведены в таблицу 1.1.

Таблица 1.1 – Сравнительная таблица методов

<i>Метод</i>	<i>Точность</i>	<i>Устойчивость к постобработке</i>	<i>Обобщающая способность</i>
Методы на основе анализа целостности изображений	Низкая/Средняя	Низкая	Низкая
Методы на основе поиска ключевых точек	Средняя	Средняя	Низкая
Однопоточная сверточная нейронная сеть	Средняя/Высокая	Средняя	Средняя
Многopotочная сверточная нейронная сеть	Высокая	Средняя/Высокая	Средняя/Высокая

На основе данных таблицы 1.1 можно сделать следующие выводы. Ме-

тоды на основе анализа целостности изображений обладают преимуществом в интерпретируемости и независимости от обучающих данных, однако их точность и робастность к постобработке остаются низкими, особенно при сжатии JPEG и изменении размера. Они рекомендуются в задачах, где важна объяснимость.

Методы на основе поиска ключевых точек (SIFT, SURF) показывают высокую точность локализации фальсификации типа копирование-вставка. Их главные недостатки — непригодность для выявления splicing и inpainting, а также вычислительная сложность, ограничивающая применение на высокоразрешенных изображениях.

Нейросетевые методы на основе CNN демонстрируют наилучшие показатели по точности локализации и робастности к постобработке, а также способны обнаруживать широкий спектр подделок. Их недостатками являются низкая интерпретируемость, высокая зависимость от размера и качества обучающей выборки.

1.4. Формализованная постановка задачи

В настоящей работе проектируется метод обнаружения фальсифицированного фрагмента на изображении с помощью свёрточной нейронной сети, а именно — двухпоточной архитектуры с слоями вниманий и адаптацией признаков. Набор данных для обучения должен быть подготовлен — должно быть установлено уникальное соответствие между входным изображением и бинарной маской. Набор данных должен быть предварительно обработан и извлечены признаки. Обученная модель используется для распознавания границ фальсифицированного фрагмента и выдачи маски локализации.

Ниже, на рисунке 1.4, представлена IDEF0-диаграмма нулевого уровня.

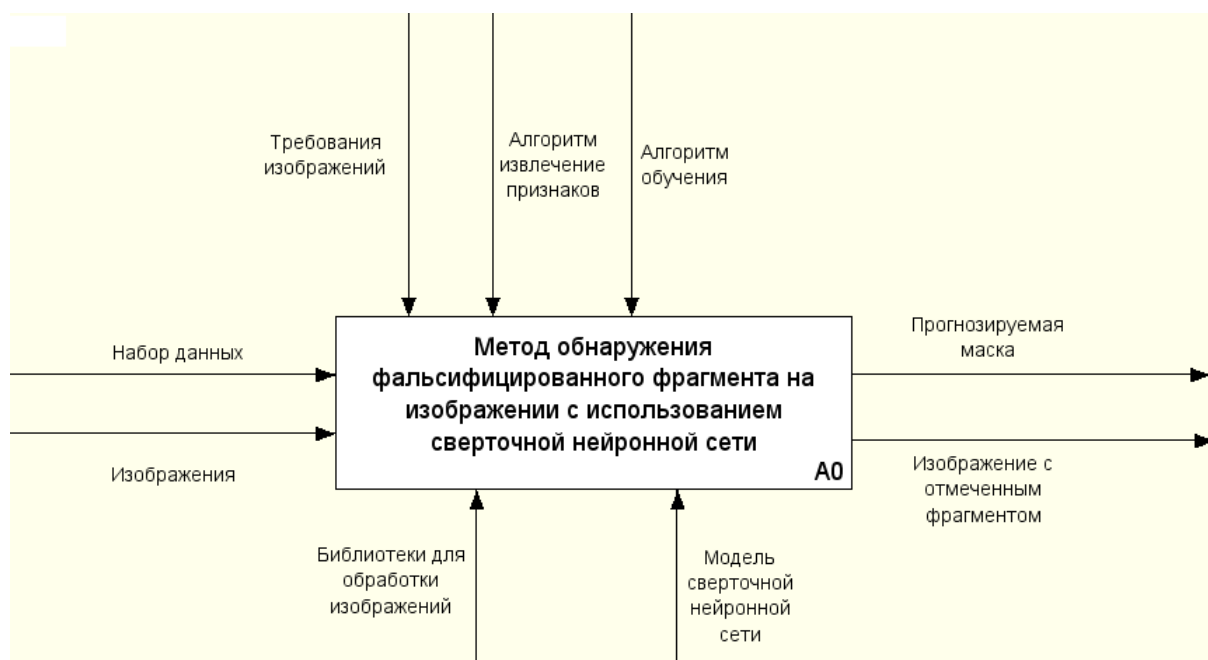


Рисунок 1.4 – IDEF0-диаграмма нулевого уровня

Вывод

В данном разделе был проведен анализ предметной области. Был проведен обзор методов обнаружения фальсификаций на изображения и их сравнение. Была представлена формализованная постановка задачи в виде IDEF0-диаграммы.

2 Конструкторский раздел

В этом разделе сформулируются требования к разрабатываемому методу и разрабатываемому программному обеспечению, описываются этапы работы алгоритма разрабатываемого метода, также рассматриваются структура разрабатываемой сверточной нейронной сети. Также описывается используемый набор данных для обучения модели и рассматривается структура программного обеспечения.

2.1. Требования и ограничения к разрабатываемому методу

К разрабатываемому методу предъявляются следующие требования.

- Метод должен базироваться на использовании сверточных нейронных сетей с необходимыми модификациями.
- Принимать на вход изображения в форматах PNG, JPG, JPEG, BMP с размером от 216×216 до 1024×1024 включительно.
- Метод должен быть применим для работы с различными типами сцен, а также различными освещением.

К разрабатываемому программному обеспечению предъявляются следующие требования.

- Возможность загрузки изображений в формате PNG, JPG, JPEG или BMP.
- Возможность обработки как RGB – изображений.
- Возможность просмотра информации по процессу обучения модели.
- Возможность просмотра значения метрик качества модели.
- Разрабатываемое ПО должно корректно реагировать на любые действия пользователя.

2.2. Описание разрабатываемого метода

2.2.1. Декомпозиция разрабатываемого метода

Ниже, на рисунке 2.1, представлена IDEF0-диаграмма разрабатываемого метода первого уровня.

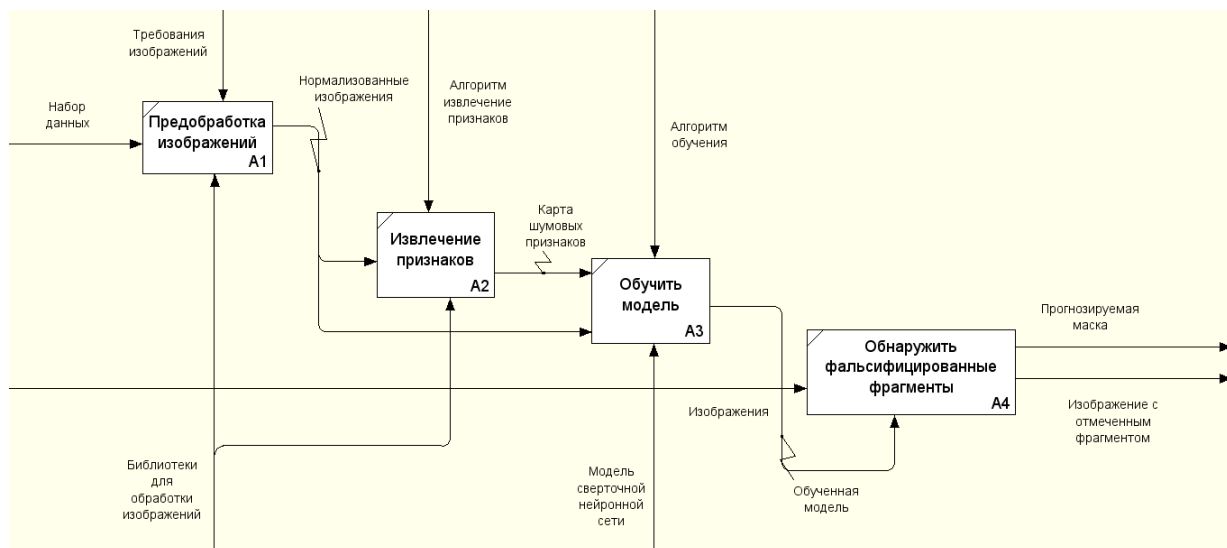


Рисунок 2.1 – IDEF0-диаграмма первого уровня

Разрабатываемый метод обнаружения фальсифицированного фрагмента на изображении состоит из четырёх последовательных этапов. Первый этап — предобработка, которая включает нормализацию входного изображения, приведение его к единому размеру и подготовку данных для последующего анализа. Второй этап — извлечение признаков — заключается в формировании карты шумовых признаков. Данный этап критически важен, поскольку именно шумовая карта несёт в себе следы неоднородности, возникающие при вставке фрагмента из другого изображения или при повторном сжатии. Алгоритм извлечения признаков спроектирован так, чтобы подчеркнуть высокочастотные составляющие, в которых маскируются артефакты фальсификации. Третий этап — обучение модели — представляет собой процесс настройки параметров двухпоточной свёрточной нейронной сети с использованием подготовленного размеченного набора данных, включая оптимизацию весов, слоёв адаптации признаков и механизмов внимания. Четвёртый этап — обнаружение фальсификации — заключается в применении обученной модели к новым изображениям для получения пиксельной маски локализации подделанного фрагмента. Такая декомпозиция позволяет чётко разделить функциональные обязанности между модулями программного обеспечения и обеспечивает гибкость.

2.2.2. Алгоритмы предобработки и извлечения признаков

Основная цель предварительной обработки изображений перед их подачей в свёрточную нейронную сеть (CNN) — повысить качество и информатив-

ность данных, чтобы улучшить детектирование артефактов, характерных для глубокого подделки. Этот этап помогает устранить шумы, нормализовать изображения, выделить ключевые признаки и минимизировать влияние вариаций освещения, масштаба и ракурса. В результате CNN может более эффективно фокусироваться на различиях между реальными и синтетическими изображениями, повышая точность обнаружения [27].

Кроме того, предобработка включает в себя приведение всех изображений к единому цветовому пространству RGB и масштабирование интенсивности пикселей к диапазону $[0,1]$, что стандартизирует входной сигнал для нейронной сети. Без такой нормализации градиенты могут стать нестабильными, а процесс обучения — замедлиться. Ниже, на рисунке 2.2, представлены шаги предобработки изображения.



Рисунок 2.2 – Шаги предобработки изображения

Для извлечения признаков из изображений используются карты шумовых признаков. Ниже на рисунку 2.3 показан алгоритм извлечения карты шумовых признаков.

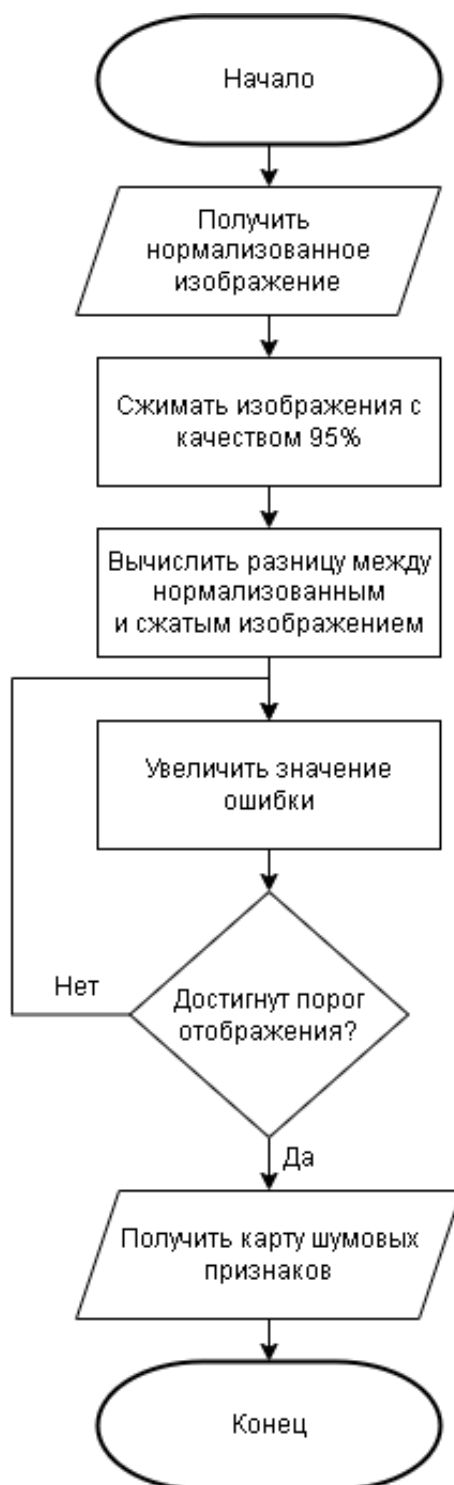


Рисунок 2.3 – Алгоритм извлечения признаков.

После завершения предварительной обработки запускается итеративный алгоритм выделения шумовых признаков. На вход подаётся нормализованное изображение. Оно сжимается с качеством 95% — это создаёт потери, характерные для сжатия. Затем вычисляется попиксельная разность между исходным нормализованным изображением и его сжатой версией. Полученная разность усиливается — значение ошибки увеличивается. После усиления проверяется, достигнут ли заданный порог отображения. Если порог не достигнут, цикл повторяется: снова вычисляется разность. Как только порог оказывается превышен, алгоритм формирует итоговую карту шумовых признаков. Таким образом, метод многократно накапливает и усиливает малые искажения, обусловленные сжатием, что позволяет выделить слабые артефакты, которые в дальнейшем могут служить характерными признаками для обнаружения.

Эта предобработка позволяет CNN сосредоточиться на анализе наиболее значимых для обнаружения подделок областей, повышая точность модели.

2.2.3. Структура разрабатываемой сверточной нейронной сети

Разрабатываемая сверточная нейронная сеть представляет собой многоуровневую архитектуру, специально спроектированную для эффективного извлечения признаков из изображений. Модель сочетает в себе последовательную обработку через блоки свёртки, слои адаптации признаков и механизмы внимания.

Выбор двухпоточной архитектуры обусловлен необходимостью разделить анализ низкоуровневых шумовых артефактов и высокоуровневого семантического содержания. Шумовой поток оперирует картой остаточных ошибок, которая чувствительна к манипуляциям, но бедна по семантике, тогда как RGB-поток сохраняет информацию о форме, цвете и текстуре объектов. Слияние этих двух потоков в глубоких слоях сети позволяет модели выявлять несоответствия, например, когда семантически корректный объект имеет неправильную шумовую характеристику.

Общая структура сети показана на рисунке 2.4.

Структура сети делится на две основные ветви, работающие с разными представлениями входных данных. Первая ветвь обрабатывает карту шумовых признаков изображения, полученную предварительно. Вторая ветвь работает

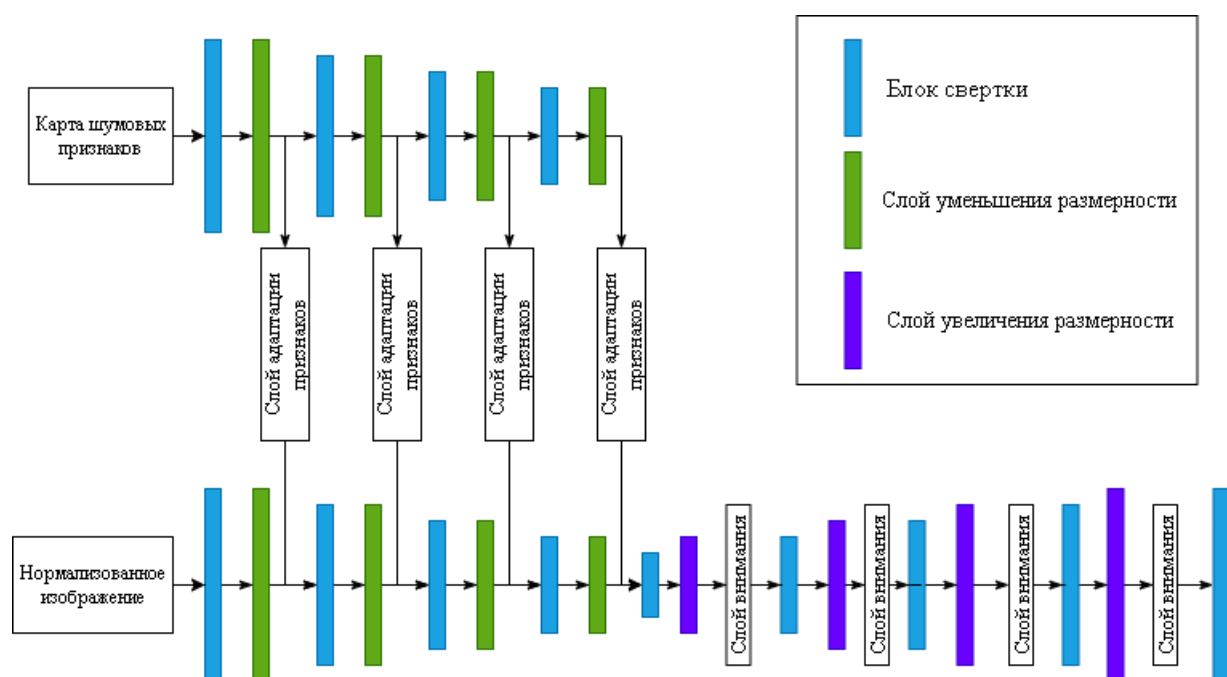


Рисунок 2.4 – Общая структура сети.

непосредственно с нормализованным исходным изображением. Такое двухпотокное проектирование позволяет сети одновременно учитывать как низкоуровневые шумовые артефакты, так и высокоуровневые семантические особенности изображения. Структура сети включает три ключевых типа компонентов: блоки свёртки, слои адаптации признаков и слои внимания, которые работают в тесной взаимосвязи.

Блок свёртки является базовым строительным элементом сети. Каждый такой блок представляет собой последовательность операций, типичную для современных CNN: свёртку 3×3 , нормализацию пакетную и функцию активации ReLU. Блоки свёртки отвечают за извлечение иерархических признаков на разных масштабах. Благодаря использованию свёрток с размером ядра 3×3 и дополнением сохраняется пространственное разрешение карт признаков, что важно для последующей точной локализации границ подделки. Нормализация пакетов ускоряет обучение и стабилизирует динамику градиентов, а нелинейность ReLU вносит необходимую нелинейность, позволяя сети аппроксимировать сложные функции.

Слои адаптации признаков — это специализированные модули, которые обеспечивают эффективное согласование и слияние признаков, поступающих из двух параллельных потоков (шумового и RGB-потока). Каждый такой слой принимает на вход карту признаков из соответствующего потока и применяет к

ней ряд преобразований: 1×1 -свёртку для изменения количества каналов, остаточное соединение и нормализацию. Основная задача этих слоёв — привести признаки из разнородных доменов (шум и естественное изображение) к единому представлению, минимизируя доменное расхождение и усиливая полезные для задачи обнаружения фальсифицированного фрагмента.

Ниже на рисунке 2.5 представлена структура слоя адаптации признаков.

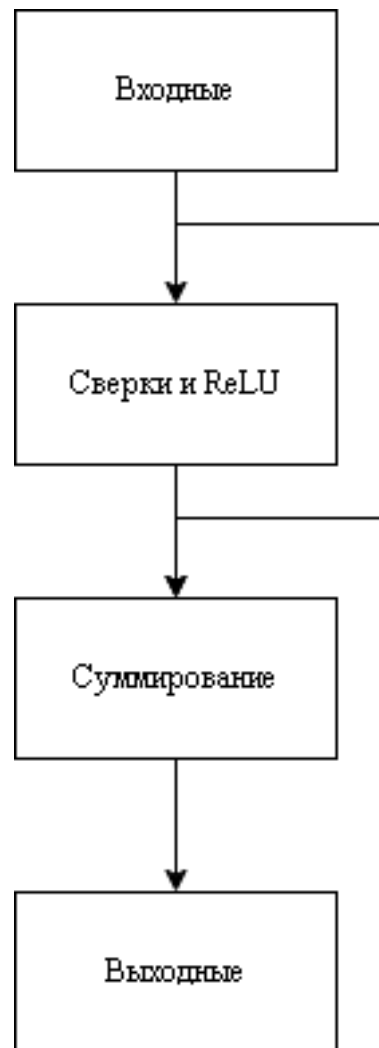


Рисунок 2.5 – Слой адаптации признаков.

Слои внимания являются важнейшим механизмом выделения релевантных признаков сети. Эти слои динамически перевешивают важность различных каналов и пространственных областей, позволяя сети концентрироваться на областях с высокой вероятностью манипуляции. Особенно эффективно слои внимания работают после слияния двух потоков, помогая выделить несоответствия между шумовыми аномалиями и семантическим содержимым. Слой внимания включает глобальный пулинг, два полносвязных слоя с активациями ReLU и

Sigmoid, а также операцию масштабирования, что позволяет динамически перераспределять веса по каналам карты признаков. Их структура представлена ниже, на рисунке 2.6.

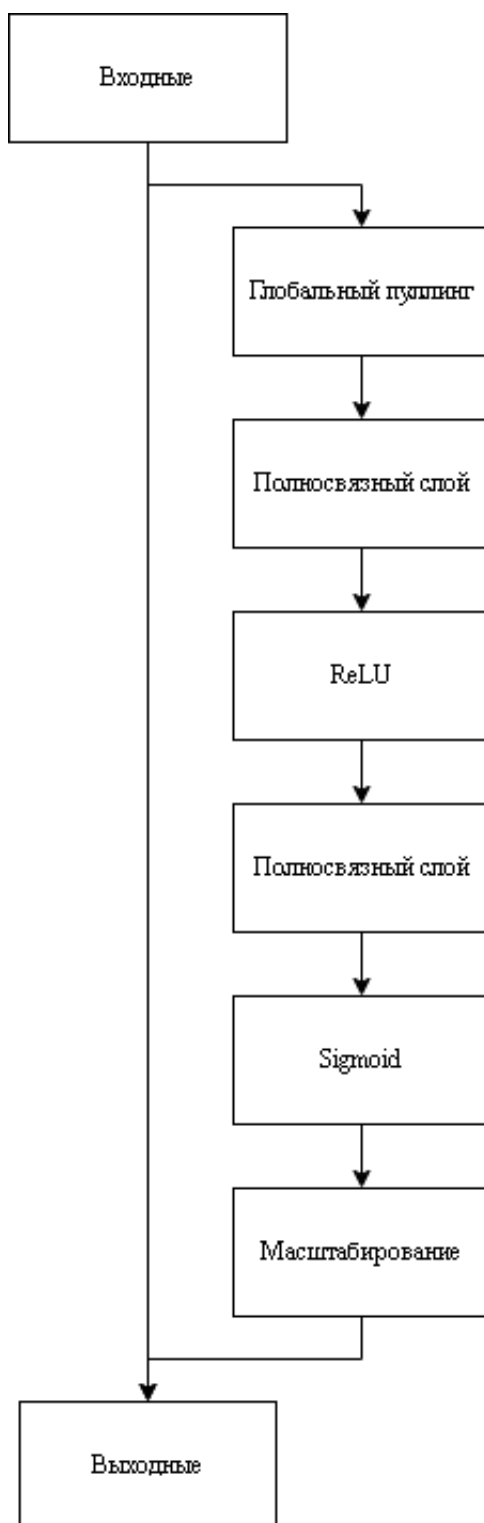


Рисунок 2.6 – Слой внимания.

Слой внимания работает следующим образом: сначала глобальный усредняющий пулинг сжимает пространственную размерность карты признаков, формируя вектор описания каналов. Затем два полносвязных слоя (с уменьшением размерности и последующим восстановлением) вычисляют веса важности для каждого канала. После применения сигмоидной функции эти веса нормализованы в диапазоне $[0,1]$ и умножаются на исходную карту признаков, усиливая значимые каналы и подавляя неинформативные. Такой механизм особенно полезен для обнаружения фальсификаций, так как поддельные области часто активируют лишь небольшое подмножество каналов.

Общая структура сети организована по принципу двух параллельных ветвей с последующим многоуровневым слиянием и декодером. Входные данные поступают в сеть по двум независимым путям. Верхний поток начинается с карты шумовых признаков, полученной предварительно. Этот поток использует блоки свёртки и слои уменьшения размерности, постепенно извлекая характерные шумовые паттерны, возникающие при JPEG-сжатии, копировании-вставке и так далее. На нескольких уровнях глубины из этого потока ответвляются признаки в слои адаптации признаков, где они приводятся в соответствие с признаками из второго потока.

Нижний поток принимает нормализованное RGB-изображение. Он также состоит из последовательности блоков свёртки и downsampling-слоёв, но сохраняет больше информации о цветовом и текстурном содержимом. После нескольких стадий извлечения признаков происходит поэтапное слияние информации из обоих потоков через слои адаптации признаков. После слияния на нескольких масштабах объединённые признаки проходят через последовательность слоёв внимания, которые уточняют представление, подавляя шум и усиливая области с подозрительными артефактами. Далее следует декодирующая часть сети, включающая слои увеличения размерности, skip-соединения с ранними уровнями энкодера и финальные свёрточные блоки. Завершается архитектура выходным слоем, формирующим карту локализации подделки с тем же пространственным разрешением, что и исходное изображение.

Использование пропущенных связей между энкодером и декодером позволяет восстанавливать мелкие пространственные детали, которые иначе теряются при многократном уменьшении разрешения. Это критически важно для точного определения границ фальсифицированного фрагмента, особенно если

он имеет небольшие размеры или сложную форму. В предложенной сети пропущенные связи организованы на четырёх масштабных уровнях, что обеспечивает высокую точность сегментации.

Такое сочетание двухпоточной обработки, модулей адаптации признаков и механизмов внимания позволяет сети эффективно обнаруживать как локальные, так и глобальные манипуляции, сохраняя при этом хорошую обобщающую способность на различных типах подделок и различных источниках изображений. Архитектура является достаточно лёгкой для обучения на современных графического процессора, но при этом обладает высокой выразительной силой благодаря продуманному взаимодействию между шумовым и визуальным доменами. В целом, предложенная структура разрабатываемой сверточной нейронной сети сочетает в себе классические принципы CNN с современными механизмами, что делает её особенно подходящей для сложной и практически значимой задачи обнаружения цифровых фальсификации изображений.

2.3. Структура разрабатываемого программного обеспечения

Разрабатываемое программное обеспечение состоит из двух модулей:

- модуль, реализующий метод обнаружения фальсифицированного фрагмента на изображениях с использованием сверточной нейронной сети;
- пользовательское приложение, производящее обнаружение фальсифицированного фрагмента на изображениях на основе разработанного метода, а так же предоставляющее возможность обучения модели, используемой в предыдущем модуле.

Модуль CNN модели

В данном модуле происходит только обучение и сохранение модели. Модуль должен быть использован либо при первоначальном обучении модели.

Результатом работы модуля является файл, содержит в себе обученную модель, т.е. ее структуру и весовые коэффициенты соответствующих связей. Это необходимо для того, чтобы не тратить время на обучение модели при повторном использовании, так как процесс обучения может занимать продолжительный период времени в зависимости от некоторых факторов.

На рисунке 2.7 представлена схема работы с модулем CNN модели.

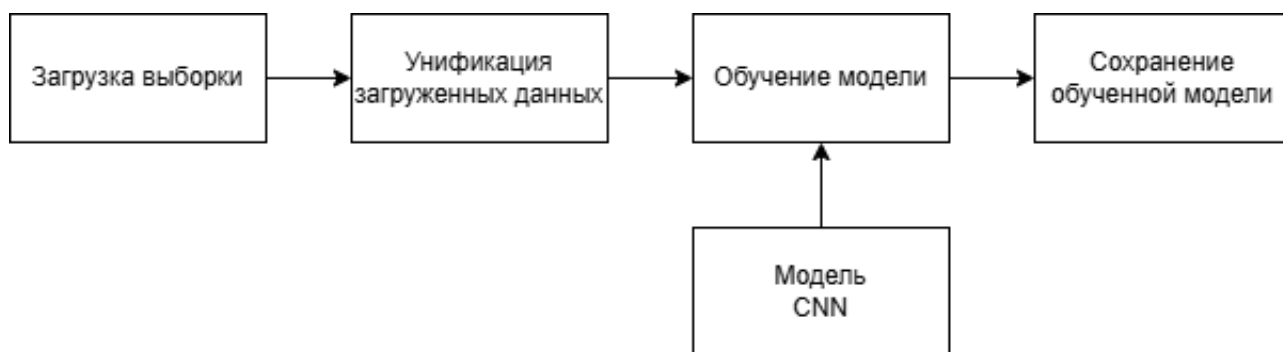


Рисунок 2.7 – Схема работы с модулем CNN модели

Модуль пользовательского приложения

Данный модуль позволяет пользователю загружать изображение для последующего анализа. Результатами работы модуля являются прогнозируемая маска и изображение с отмеченным фрагментом.

На рисунке 2.8 представлена схема работы с модулем пользовательского приложения.

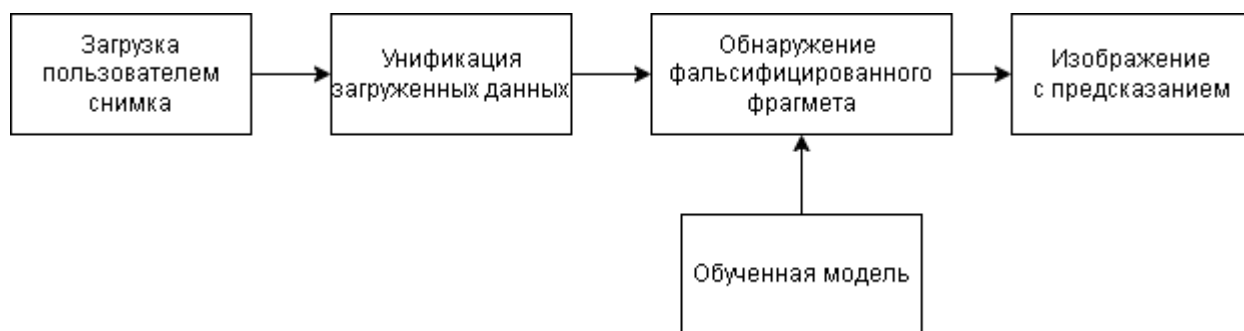


Рисунок 2.8 – Схема работы с модулем пользовательского приложения

2.4. Набор данных для обучения модели

Для обучения и оценки модели использовался широко известный в области цифровой криминалистики набор данных CASIA v2.0. Данный датасет был считается одним из наиболее крупных и сложных публичных наборов данных для задач обнаружения и локализации различных типов манипуляций с изображениями[28].

CASIA v2.0 содержит 5123 поддельных изображений и соответственные бинарные маски. Поддельные изображения включают два основных типа манипуляций: сору-move (копирование и вставка внутри одного изображения) и splicing (вставка объектов из других изображений). Изображения имеют различ-

ное разрешение и представлены в нескольких форматах: PNG, JPEG и TIFF. Важной особенностью датасета является наличие постобработки поддельных областей (размытие, сжатие, поворот, масштабирование), что делает задачу локализации значительно более сложной и приближенной к реальным условиям.

Применение CASIA v2.0 позволило модели обучаться на разнообразных сценариях подделок, различных текстурах, освещении и постобработках, что существенно повышает обобщающую способность сети. Перед обучением изображения проходили этап нормализации и предварительной аугментации для повышения устойчивости модели к вариациям условий съёмки и сжатия.

Использование именно этого датасета обеспечивает возможность объективного сравнения результатов предлагаемой модели с существующими современными подходами в области обнаружения цифровой фальсификации изображений.

Вывод

В этом разделе были сформулированы требования к разрабатываемому методу и разрабатываемому программному обеспечению и описаны этапы работы алгоритма разрабатываемого метода, также была рассматривана структура разрабатываемой сверточной нейронной сети. Был описан используемый набор данных для обучения модели и была рассмотрена структура программного обеспечения.

3 Технологический раздел

В этом разделе приводится выбор языка программирования и средства программирования и рассматриваются необходимые библиотеки, которые используются для разработки программного обеспечения. Также описывается формат входных и выходных данных. Также приводятся описание пользовательского интерфейса и руководство пользователя для установки и использования программного обеспечения.

3.1. Выбор средств реализации программного обеспечения

3.1.1. Выбор языка программирования

Для реализации программного обеспечения был использован язык программирования Python[29]. Этот выбор обусловлен следующими причинами:

- это язык программирования высокого уровня, имеет простой синтаксис;
- наличие библиотек с открытым исходным кодом, предоставляющих полный набор инструментов для предварительной обработки изображений и построения сетей СНН;
- имеется навыки использования данного языка программирования, что позволяет сократить время написания программы.

Кроме того, Python является стандартом в области глубокого обучения и компьютерного зрения: подавляющее большинство исследовательских проектов и промышленных решений в этой сфере реализованы именно на Python. Это обеспечивает широкую поддержку сообщества, обилие готовых примеров кода и возможность интеграции с другими инструментами. Ещё одним важным преимуществом является кроссплатформенность: код, написанный на Python, может выполняться на Windows, Linux и macOS без существенных изменений.

3.1.2. Выбор среды программирования

Для разработки программы использовалась среда Visual Studio Code [30] в качестве среды разработки по следующим причинам:

- доступна бесплатная версия;

- имеет множество утилит, упрощающих написание кода;
- интеграция с системой контроля версий Git и инструментами разработки;
- имеются навыки программирования при помощи данной среды, что сокращает время написания программы.

Visual Studio Code также предлагает богатую экосистему расширений: для работы с Python, для форматирования кода, для проверки синтаксиса и для работы с Jupyter Notebooks. Встроенный терминал и отладчик позволяют быстро запускать скрипты и находить ошибки. Благодаря низкому потреблению ресурсов VS Code комфортно работает даже на устройствах с ограниченной памятью.

Библиотека `torch` (PyTorch)[31] используется для создания сверточной нейронной сети, потому что она предоставляет все необходимые инструменты, такие как базовые слои для создания сетевых архитектур, функции активации, оптимизаторы и функции потерь для обучения. В частности, PyTorch поддерживает вычисления на графических процессорах для ускорения обучения и автоматически рассчитывает градиенты, делая процесс обратного распространения простым и гибким. Это помогает сосредоточиться на разработке модели.

Дополнительным преимуществом PyTorch является его динамический вычислительный граф, который позволяет изменять архитектуру сети на лету и упрощает отладку. В отличие от статических графов TensorFlow 1.x, динамический граф делает код более интуитивным и похожим на обычные программы на Python.

Для создания пользовательского интерфейса используется библиотека Streamlit. Streamlit[32] — мощная и простая библиотека Python с открытым исходным кодом для создания пользовательских интерфейсов для приложений Data Science и машинного обучения. Она превращает обычные скрипты Python в интерактивные веб-приложения, не требуя знаний фронтенда. С помощью Streamlit легко создавать виджеты и визуализации данных (таблицы, диаграммы). При каждом взаимодействии пользователя весь скрипт запускается заново, сохраняя логику работы.

Streamlit был выбран также из-за минимального объема кода, необходимого для создания полноценного интерфейса. Для реализации загрузки изображения, отображения результатов и настройки параметров потребовалось менее

200 строк кода, что значительно меньше, чем при использовании традиционных фреймворков (Flask, Django) с отдельной фронтенд-частью. Это ускорило разработку и позволило сосредоточиться на основной функциональности метода.

3.2. Формат входных и выходных данных

На этапе обучения модели входными данными является набор данных, представленный парой «исходное изображение — пиксельная маска разметки». Каждое исходное изображение представляет собой цветную фотографию в формате JPG, JPEG, PNG и TIF. Пиксельная маска разметки служит эталоном и представляет собой одноканальное изображение в градациях серого формата PNG. Размер исходного изображения и размер соответствующей пиксельной маски разметки должны быть одинаковыми.

Это требование синхронизации размеров критически важно, поскольку сеть обучается попиксельному сопоставлению: каждый пиксель входного изображения должен иметь соответствующую метку в маске. В случае несовпадения размеров обучение становится невозможным или приводит к ошибкам. Все изображения в используемом датасете CASIA v2.0 удовлетворяют этому условию. Маски представлены в бинарном виде: белый цвет (255) соответствует фальсифицированному фрагменту, чёрный (0) — подлинной области.

После завершения этапа обучения модели обученная модель будет сохранена для использования на этапе распознавания.

На этапе распознавания входными данными являются цветное изображение в формате JPG, JPEG, PNG и TIF размером от 216x216 до 1024x1024 и обученная модель. Возвращаемые результаты представляют собой предсказание об изображении и обрамляющие окна, окружающие измененные области, отображаемые в пользовательском интерфейсе.

Диапазон размеров (от 216×216 до 1024×1024) выбран эмпирически: слишком маленькие изображения (менее 216 пикселей по меньшей стороне) не содержат достаточного количества деталей для надёжной локализации, а слишком большие (свыше 1024×1024) требуют значительных вычислительных ресурсов и могут не поместиться в видеопамять GPU.

3.3. Структура разрабатываемого программного обеспечения

Разрабатываемый программный продукт состоит из следующих частей.

1. Модуль предобработки изображений — отвечает за предварительную обработку и извлечение признаков из изображений.
2. Модуль обнаружения фальсифицированного фрагмента — отвечает за обнаружение фальсифицированного фрагмента на изображении.
3. Модуль пользовательского интерфейса — отвечает за предоставление пользовательского интерфейса.
4. Модуль сохранения и загрузки модели — отвечает за сохранение обученной модели и загрузку сохраненной модели для использования на этапе обнаружения.

Ниже, на рисунке 3.1 представлена диаграмма структуры разрабатываемого программного обеспечения.

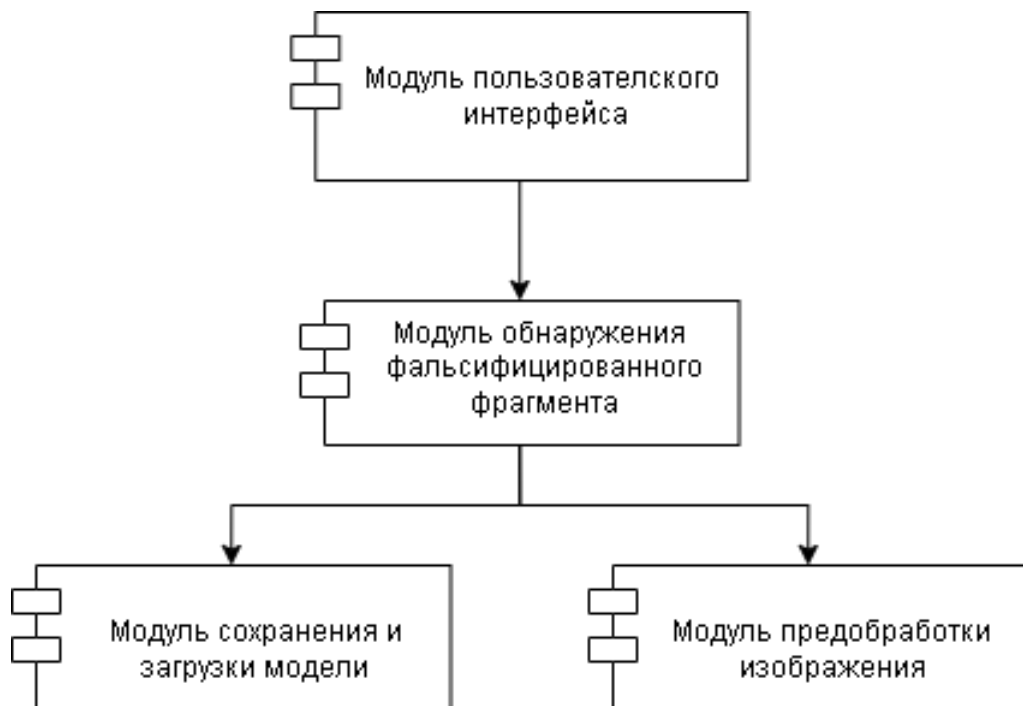


Рисунок 3.1 – Структура разрабатываемого программного обеспечения.

3.4. Руководство пользователя

Для запуска разработанного программного обеспечения требуется установить интерпретатор для Python и используемые библиотеки. Используемые

в разработке библиотеки, которые необходимы для запуска ПО, приведены в файле `requirements.txt`, который находится в корневом каталоге проекта. С помощью пакетного менеджера `pip` все зависимости можно установить или обновить, запустив в терминале команду, приведенную в листинге 3.1.

Листинг 3.1 – Установка всех необходимых библиотек

```
1 pip install -r requirements.txt
```

Для обучения модели на наборе данных, необходимо запустить скрипт `training.py` через интерфейс среды разработки или выполнив команду в терминале, как показано в листинге 3.2.

Листинг 3.2 – Команда для запуска обучения модели

```
1 python training.py
```

Обучение модели может выполняться как на локальной машине с поддержкой GPU, так и в облачной среде Kaggle. Для локального запуска с использованием графического процессора необходимо предварительно установить CUDA-совместимую версию PyTorch и драйверы NVIDIA.

Если локальный GPU отсутствует или его производительность недостаточна, рекомендуется использовать платформу Kaggle, которая предоставляет бесплатный доступ к GPU (NVIDIA P100 или T4). Для этого нужно загрузить проект в виде Kaggle Notebook, включить в настройках ускоритель GPU, загрузить датасет CASIA v2.0 (например, как отдельный Dataset или через добавление файлов) и выполнить ту же команду в ячейке ноутбука. Kaggle автоматически управляет средой, поэтому нет необходимости вручную настраивать драйверы. После завершения обучения полученная модель может быть скачана и использована в локальном приложении. Такой подход особенно полезен при ограниченных вычислительных ресурсах или при необходимости быстрого экспериментирования с гиперпараметрами.

После обучения модели пользователь может использовать ее через пользовательский интерфейс, используя команды, представленные в листинге 3.3.

Листинг 3.3 – Команда для запуска приложения

```
1 python -m streamlit run app.py
```

После выполнения этой команды в терминале появится локальный URL (обычно `http://localhost:8501`), который необходимо открыть в веб-браузере. Приложение будет работать до тех пор, пока терминал не будет закрыт или

не будет прервано выполнение (Ctrl+C). Streamlit автоматически перезагружает приложение при изменении исходного кода, что удобно для отладки. Для остановки сервера достаточно нажать Ctrl+C в терминале.

В результате разработанное программное обеспечение может предсказывать об изображении и показать фальсифицированные фрагменты в изображении.

3.5. Интерфейс пользователя

Пользовательский интерфейс, который представлен на рисунке 3.2, был разработан с помощью библиотеки streamlit.

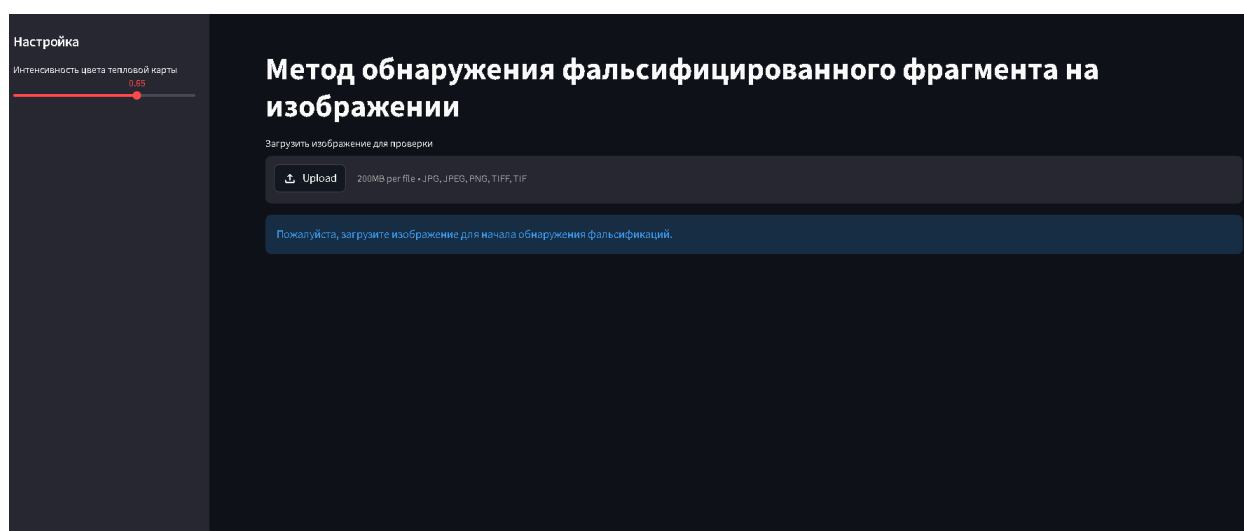


Рисунок 3.2 – Интерфейс пользователя.

Интерфейс разделен на две основные части: панель настроек и область загрузки изображений. В разделе настроек пользователи могут настроить интенсивность цвета тепловой карты. Основная часть приложения сосредоточена на функции загрузки изображений, поддерживая перетаскивание файлов или ручной выбор в различных форматах для анализа и визуального отображения областей с искажениями в виде тепловых карт, бинарных масок.

После выбора пользователем файла изображения модель обработает его и выдаст результат. Если изображение не содержит фальсифицированного фрагмента, будет выдано сообщение, как показано на рисунке 3.3.

Если загруженный файл изображения содержит фальсифицированные фрагменты, результатом будет изображение, содержащее ограничивающие рамки, указывающие области изображения, которые могут быть сфальсифицированы, процент сфальсифицированных пикселей и координаты этих ограничиваю-

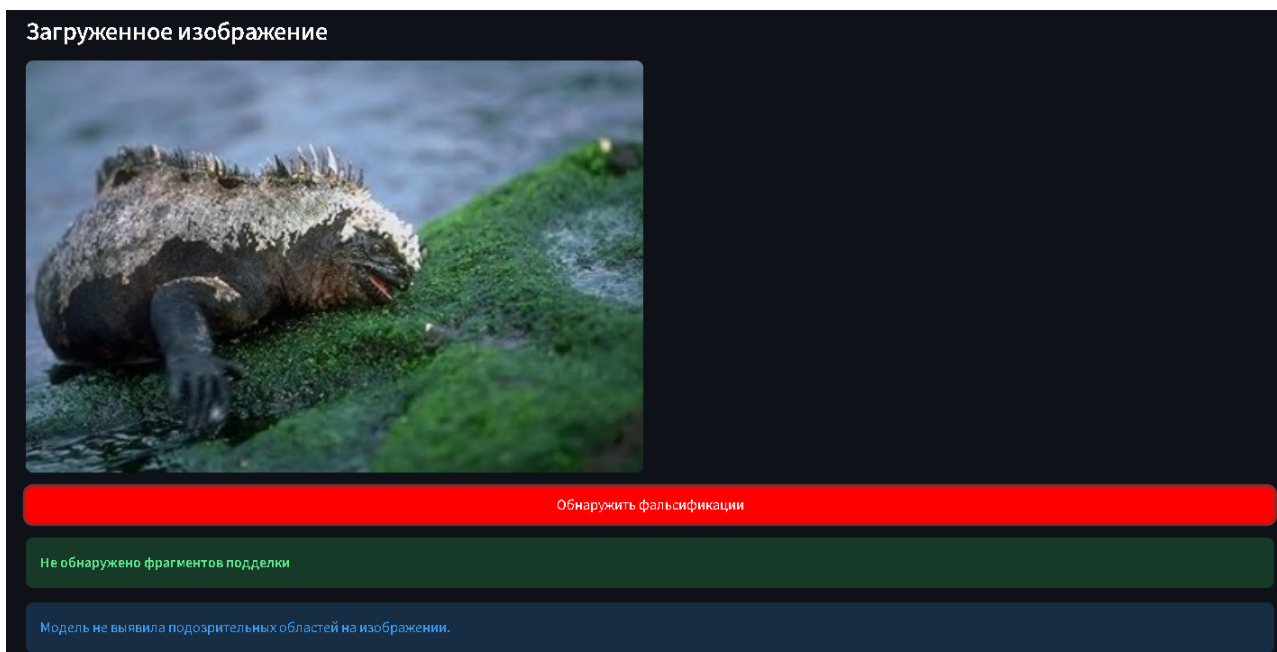


Рисунок 3.3 – Пример обнаружения при загрузке оригинального файла изображения

щих рамок. Ниже, на рисунке 3.4, приведён пример файла изображения, который имеет сфальсифицированные фрагменты.

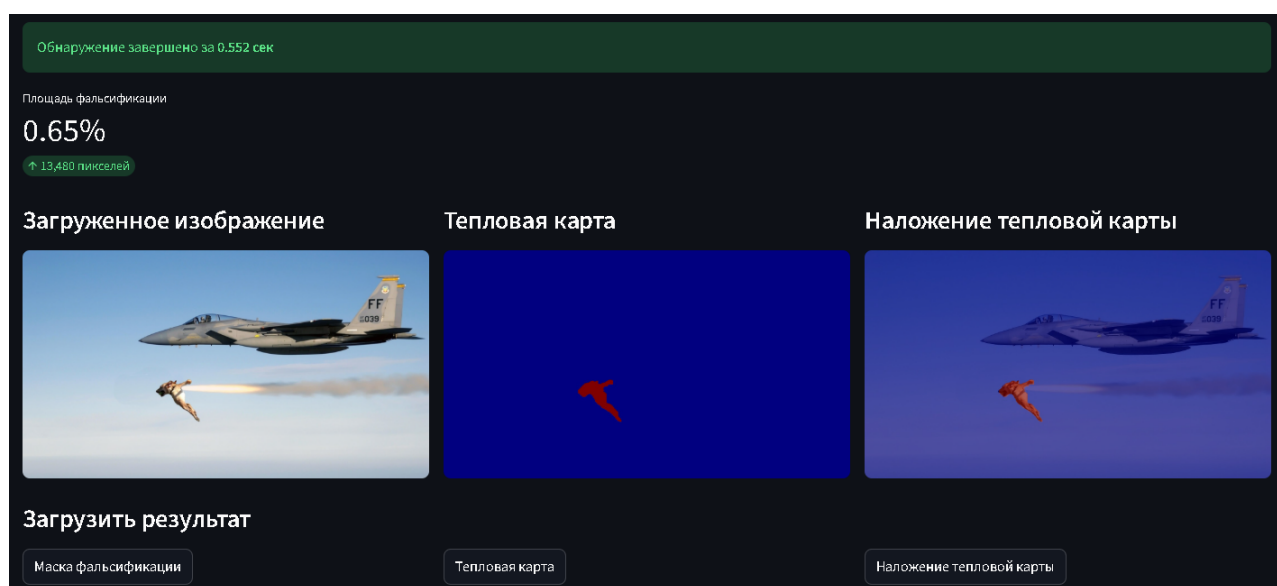


Рисунок 3.4 – Пример обнаружения при загрузке файла изображения, который содержит фальсифицированные фрагменты

Вывод

В этом разделе были приведены выбор языка программирования и средства программирования и рассмотрены необходимые библиотеки, которые ис-

пользуются для разработки программного обеспечения. Также был описан формат входных и выходных данных. Также были приведены описание пользовательского интерфейса и руководство пользователя для установки и использования программного обеспечения.

4 Исследовательский раздел

4.1. Метрики оценки качества модели

Поскольку задача обнаружения фальсифицированного фрагмента на изображении формулируется как задача сегментации на уровне пикселей, оценка качества модели должна учитывать как классификационную точность, так и пространственное соответствие предсказанных и истинных областей сфальсифицированного. В связи с этим для оценки качества метода используется комплекс метрик, специализированных для задач локализации аномалий.

Базой для вычисления всех показателей служат следующие понятия, определяемые на уровне каждого пикселя тестовой выборки:

- TP (англ. True Positive) — количество пикселей, корректно классифицированных как принадлежащих области сфальсифицированных;
- TN (англ. True Negative) — количество пикселей, корректно классифицированных как принадлежащих фону;
- FP (англ. False Positive) — количество пикселей, ошибочно отнесённых к области сфальсифицированных;
- FN (англ. False Negative) — количество пикселей области сфальсифицированных, пропущенных моделью.

Точность на уровне пикселей отражает долю правильно классифицированных пикселей от общего числа пикселей в тестовой выборке:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}. \quad (4.1)$$

Данная метрика удобна для общей оценки, однако при сильном дисбалансе классов (область подделки обычно занимает менее 5-10 % площади изображения) она может давать завышенные результаты за счёт доминирования класса фона.

Для более объективной оценки качества выделения целевой области используются точность (Precision) и полнота (Recall):

$$Precision = \frac{TP}{TP + FP}. \quad (4.2)$$

$$Recall = \frac{TP}{TP + FN}. \quad (4.3)$$

На основе этих показателей вычисляется F1-мера, представляющая собой гармоническое среднее точности и полноты:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}. \quad (4.4)$$

F1-мера устойчива к дисбалансу классов и широко применяется в задачах обнаружения локальных артефактов, поскольку учитывает как пропущенные ложные пиксели, так и ложные срабатывания на фоне.

Ключевой метрикой пространственного соответствия является коэффициент пересечения над объединением (англ. Intersection over Union, IoU):

$$IoU = \frac{TP}{TP + FP + FN}. \quad (4.5)$$

Метрика IoU измеряет степень перекрытия предсказанной маски подделки с эталонной разметкой и является стандартом де-факто в задачах сегментации. Значение $IoU = 1$ указывает на идеальное совпадение границ, а $IoU=0$ — на полное отсутствие пересечения.

В отличие от F1-меры, IoU более строго штрафует за плохое совпадение границ: например, если предсказанная маска полностью покрывает истинную, но при этом значительно больше её по площади, F1-мера может оставаться высокой, а IoU резко падает. Поэтому в задачах точной локализации фальсифицированных фрагментов рекомендуется использовать обе метрики совместно. В настоящем исследовании итоговая оценка качества модели производится по средним значениям F1 и IoU на тестовой выборке.

Все метрики вычисляются на уровне пикселей для каждой тестовой пары «изображение – маска», после чего усредняются по всей тестовой выборке. Такой комплексный подход (F1-мера и IoU) обеспечивает всестороннюю оценку способности сверточной нейронной сети локализовывать фальсифицированные фрагменты при различных объёмах и конфигурациях входных данных. В дальнейшем эти две метрики будут использоваться как основные для анализа результатов всех экспериментов.

4.2. Технические характеристики

Технические характеристики устройства, на котором выполнялись исследования разработанного метода:

- операционная система Window 10 Home Single Language [33];
- память 8 Гб;
- процессор 11th Gen Intel(R) Core(TM) i5-1135G7 2.42 ГГц, 4 ядра [34].

Во время выполнения исследований устройство было подключено к сети электропитания, нагружено приложениями окружения и самой системой замера.

Для проведения исследований разработанного метода используются выборки из набора данных Casia v2. Количество изображений в каждой раз используется одинаковое. Исходная выборка разбивается на обучающую (80% объема) и тестовую (20%) выборки.

Как отмечалось раньше, метрики F1-меры и IoU используются в качестве метрики для оценки качества классификатора.

4.3. Влияние слоя внимания на качество модели

Целью данного исследования является изучение эффективности использования слоев внимания в реализованной модели сверточной нейронной сети.

Для проведения этого исследования было выполнено сравнительное тестирование двух конфигураций модели: с использованием слоев внимания и без них. Все остальные компоненты архитектуры оставались неизменными. Эксперимент проводился на одной и той же тестовой выборке, что обеспечило объективность сравнения. Результаты эксперимента представлены в таблице 4.1.

Таблица 4.1 – Зависимость точности от конфигураций

<i>Конфигурация</i>	<i>Значение F1</i>	<i>Значение IoU</i>
С использованием слоев внимания	0.9042	0.8252
Без использования слоев внимания	0.6784	0.5133

Примеры результатов обнаружения модели при различных конфигурациях слоев внимания представлены на рисунке 4.1.



Рисунок 4.1 – Зависимость качества модели от конфигураций слоев внимания.

На основании полученных результатов можно сделать вывод, что добавление слоев внимания приводит к существенному приросту обеих ключевых метрик. Данный результат подтверждает высокую эффективность механизма внимания для задачи обнаружения фальсифицированных фрагментов. Поскольку область подделки обычно занимает малую долю изображения, большинство каналов признаков содержит информацию о нормальном фоне. Без механизма внимания модель «размазывает» внимание по всем каналам, что приводит к высокому уровню ложноположительных срабатываний (FP) и пропуску мелких деталей подделки (FN). Слой внимания позволяет модели фокусироваться на высокочастотных артефактах, характерных для фальсифицированных областей. Это особенно важно при решении задачи сегментации с сильным дисбалансом классов. Таким образом, использование слоев внимания значительно повышает качество модели, делая её более чувствительной к локальным артефактам подделки.

4.4. Влияние слоя адаптации признаков на качество модели

Целью данного исследования является изучение эффективности использования слоев адаптации признаков в реализованной модели сверточной нейронной сети.

Для проведения этого исследования было выполнено сравнительное тестирование трёх конфигураций модели: без использования слоев адаптации признаков, со слоями адаптации в виде единых блоков свёртки и со слоями адаптации в виде остаточных блоков. Все остальные компоненты архитектуры оставались неизменными. Эксперимент проводился на одной и той же тестовой выборке. Результаты эксперимента представлены в таблице 4.2.

Таблица 4.2 – Зависимость точности от конфигураций

<i>Конфигурация</i>	<i>Значение $F1$</i>	<i>Значение IoU</i>
Без использования слоев адаптации признаков	0.6829	0.6011
Слои адаптации признаков как единые блоки свёртки	0.7653	0.7139
Слои адаптации признаков как остаточные блоки	0.9042	0.8252

Примеры результатов обнаружения модели при различных конфигурациях слоев адаптации признаков представлены на рисунке 4.2.

На основании полученных результатов можно сделать вывод, что отказ от слоев адаптации признаков приводит к значительному снижению качества, использование простых сверточных блоков даёт умеренное улучшение, наилучшие результаты достигаются при реализации слоев адаптации в виде остаточных блоков. Основная причина такого различия заключается в природе признаков, поступающих из ветви извлечения признаков. Эти признаки содержат высокочастотные детали, которые при прямом объединении с семантическими картами основной ветви создают значительные помехи. Это приводит к размытию границ предсказанной маски и высокому уровню ложноположительных результатов на фоне. При использовании единых сверточных блоков качество улучшается, но всё ещё наблюдается потеря мелких деталей. Остаточные блоки обеспечивают чёткие, точные границы маски и минимальное количество ложных сра-



Рисунок 4.2 – Зависимость качества модели от конфигураций слоев адаптации признаков.

батываний даже на сложных текстурах фона. Таким образом, слои адаптации признаков, реализованные в виде остаточных блоков, являются оптимальным решением. Они эффективно фильтруют помехи, сохраняют важную высокочастотную информацию и обеспечивают наилучшее качество локализации фальсифицированных фрагментов при сильном дисбалансе классов.

4.5. Влияние объема передаваемых данных на качество работы метода

Целью данного исследования является изучение зависимости точности метода обнаружения фальсифицированного фрагмента от объема передаваемых данных. Поскольку в рамках используемой архитектуры сверточной нейронной сети основным фактором, определяющим объем обрабатываемой информации, выступает пространственное разрешение входного изображения, в эксперименте рассматривались изображения различных размеров: 384×256 , 640×480 и 800×600 пикселей. Большее число пикселей соответствует большему объему данных, поступающих на вход сети.

Тестирование проводилось на единой тестовой выборке. Все компоненты модели, включая слои внимания и адаптации признаков, оставались неизменными. В качестве оцениваемых показателей использовались усредненные по каждой размерной группе значения F1-меры и IoU. Результаты эксперимента представлены ниже в таблице 4.3.

Таблица 4.3 – Зависимость точности от размеров входного изображения

<i>Размер изображения</i>	<i>Значение F1</i>	<i>Значение IoU</i>
384×256	0.9305	0.8887
640×480	0.9270	0.8792
800×600	0.5834	0.5190

На основании полученных результатов можно сделать вывод, что при умеренном объеме передаваемых данных (разрешения 384×256 и 640×480) модель демонстрирует стабильно высокое качество локализации: средняя F1-мера превышает 0,92, а средний IoU — 0,87. Это свидетельствует о том, что в пределах исследованного диапазона, характерного для многих практических сценариев, эффективность метода практически не зависит от объема данных. Однако при переходе к изображениям большего размера 800×600, то есть при значительном увеличении объема передаваемой информации, наблюдается резкое падение обеих метрик: средняя F1-мера снижается до 0,5834, а средний IoU — до 0,5190. Такое ухудшение объясняется тем, что текущая архитектура сети ориентирована на определенный пространственный масштаб признаков, и при избыточном объеме данных высокочастотные артефакты подделки либо маскируются обилием фоновых деталей, либо требуют более глубокого анализа, не обеспеченного существующей конфигурацией.

Таким образом, проведенный эксперимент подтверждает, что реализованный метод сохраняет высокую эффективность для объемов передаваемых данных, соответствующих разрешениям приблизительно до 640×480. Дальнейшее повышение качества на данных большего объема может быть достигнуто путем модификации архитектуры с целью адаптации к высокоразрешающим изображениям.

4.6. Влияние качества изображения на качество модели

Целью данного исследования является изучение влияния качества изображений на точность разработанного метода.

Для проверки устойчивости модели был проведен эксперимент по сжатию входных изображений по стандарту JPEG. Поскольку в реальных условиях изображения часто подвергаются сжатию, важно оценить, как изменение коэффициента качества влияет на точность локализации. Исследование проводи-

лось при варьировании качества от 0.85 до 1, а результаты по метрикам F1-score и IoU представлены на рисунке 4.3.

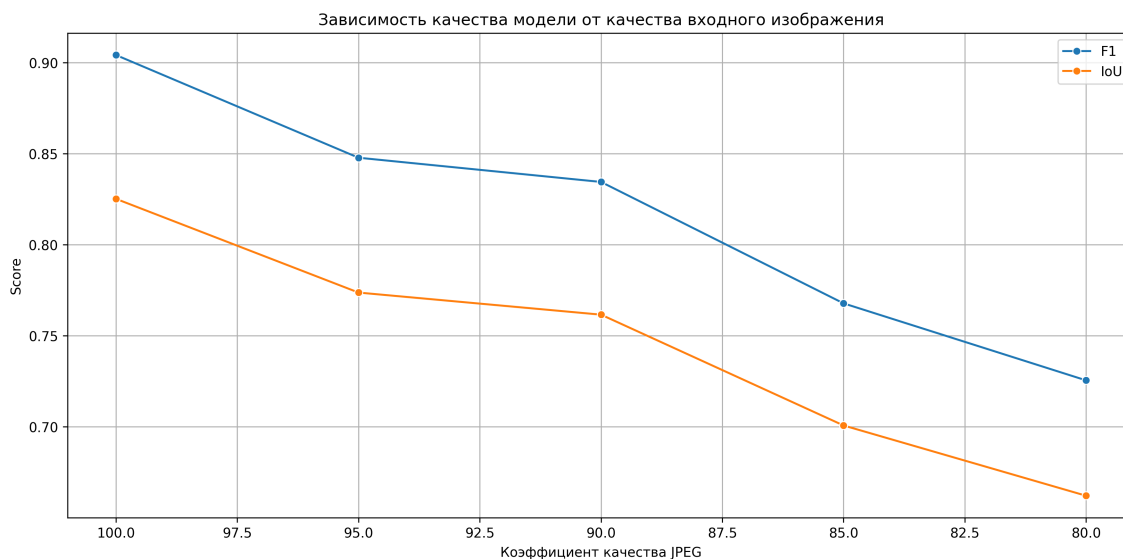


Рисунок 4.3 – Зависимость качества модели от качества изображений.

На основании полученных результатов можно сделать вывод, что точность разработанного метода существенно зависит от уровня сжатия JPEG входного изображения. При высоком качестве изображения (коэффициент JPEG равен 100) модель демонстрирует наилучшие результаты. Однако с увеличением степени сжатия (уменьшением коэффициента качества JPEG) наблюдается устойчивое снижение обеих метрик. Данная зависимость свидетельствует о достаточной чувствительности модели к потерям информации, вызванным JPEG-компрессией, которая является одним из наиболее распространённых видов постобработки в реальных условиях. Тем не менее, даже при сильном сжатии модель сохраняет приемлемый уровень точности локализации, что подтверждает её определённую робастность. Полученные результаты указывают на необходимость дальнейшего совершенствования архитектуры, в частности, путём введения дополнительных механизмов устойчивости к JPEG-артефактам.

Вывод

В ходе исследования разработанной сверточной нейронной сети были выбраны и обоснованы основные метрики оценки качества локализации фальсифицированного фрагмента — F1-мера и IoU. Эксперименты показали, что использование слоя внимания значительно повышает точность модели. Наилучшие результаты достигаются при применении слоёв адаптации признаков в ви-

де остаточных блоков, которые эффективно фильтруют шум и сохраняют важную информацию. Исследование влияния объема передаваемых данных показало стабильные высокие результаты на изображениях разрешений до 640×480 и заметное снижение качества при переходе к большим объемам (800×600). Кроме того, установлено, что качество работы модели сильно зависит от степени сжатия JPEG: при коэффициенте 100 достигаются максимальные значения метрик, однако даже при сильном сжатии (JPEG = 80) модель сохраняет приемлемую точность. Полученные результаты подтверждают эффективность предложенного подхода и указывают на необходимость дальнейшего повышения робастности к постобработке изображений.

ЗАКЛЮЧЕНИЕ

В рамках настоящей работы был разработан, реализован и исследован метод обнаружения фальсифицированного фрагмента на изображении с использованием сверточной нейронной сети. Все поставленные задачи были выполнены.

Был проведен анализ предметной области. Были приведены обзор и сравнение существующих методов обнаружения фальсифицированного фрагмента на изображении. Была представлена формализованная постановка задачи в виде IDEF0-диаграммы.

Были описан метод обнаружения фальсифицированного фрагмента на изображении с использованием сверточной нейронной сети, приведены IDEF0-диаграммы метода и схемы его алгоритмов.

Были приведены выбор языка программирования и средства программирования и рассмотрены необходимые библиотеки, которые используются для разработки программного обеспечения. Также был описан формат входных и выходных данных. Также были приведены описание пользовательского интерфейса и руководство пользователя для установки и использования программного обеспечения.

Были описаны технические характеристики устройства для проведения исследований и приведены исследования разработанного метода

Для реализованного метода можно предложить следующие развития:

- ускорение работы метода.
- добавление возможности обнаружения разных областей фальсификации.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Farid H.* Digital Image Forensics // Scientific American. — 2008. — Июнь. — Т. 298. — С. 66—71. — DOI: 10.1038/scientificamerican0608-66.
2. *Verdoliva L.* Media Forensics and DeepFakes: An Overview // IEEE Journal of Selected Topics in Signal Processing. — 2020. — Т. 14, № 5. — С. 910—932. — DOI: 10.1109/JSTSP.2020.3002101.
3. *Birajdar G., Mankar V.* Digital image forgery detection using passive techniques: A survey // Digital Investigation. — 2013. — Окт. — Т. 10. — С. 226—245. — DOI: 10.1016/j.diin.2013.04.007.
4. *Lukás J., Fridrich J., Goljan M.* Detecting digital image forgeries using sensor pattern noise // Proceedings of SPIE - The International Society for Optical Engineering. — 2006. — Февр. — Т. 6072. — С. 362—372. — DOI: 10.1117/12.640109.
5. *Qureshi M. A., Deriche M.* A bibliography of pixel-based blind image forgery detection techniques // Signal Processing: Image Communication. — 2015. — Т. 39. — С. 46—74. — ISSN 0923-5965. — DOI: <https://doi.org/10.1016/j.image.2015.08.008>. — Режим доступа, URL: <https://www.sciencedirect.com/science/article/pii/S0923596515001393>.
6. *Piva A.* An Overview on Image Forensics // International Scholarly Research Notices. — 2013. — Т. 2013, № 1. — С. 496701. — DOI: <https://doi.org/10.1155/2013/496701>. — Режим доступа, URL: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2013/496701>.
7. *Cozzolino D., Verdoliva L.* Noiseprint: a CNN-based camera model fingerprint // CoRR. — 2018.
8. Detection of GAN-Generated Fake Images over Social Networks / F. Marra [и др.] // — 04.2018. — С. 384—389. — DOI: 10.1109/MIPR.2018.00084.
9. Deep Learning for Multimedia Forensics / I. Amerini [и др.] // Foundations and Trends in Computer Graphics and Vision. — 2021. — Авг. — Т. 12. — С. 309—457. — DOI: 10.1561/06000000096.
10. Determining Image Origin and Integrity Using Sensor Noise / M. Chen [и др.] // IEEE Transactions on Information Forensics and Security. — 2008. — Т. 3, № 1. — С. 74—90. — DOI: 10.1109/TIFS.2007.916285.

11. *Bianchi T., Piva A.* Detection of Nonaligned Double JPEG Compression Based on Integer Periodicity Maps // IEEE Transactions on Information Forensics and Security. — 2012. — Т. 7, № 2. — С. 842—848. — DOI: 10.1109/TIFS.2011.2170836.
12. Image Forgery Localization via Fine-Grained Analysis of CFA Artifacts / P. Ferrara [и др.] // IEEE Transactions on Information Forensics and Security. — 2012. — Т. 7, № 5. — С. 1566—1577. — DOI: 10.1109/TIFS.2012.2202227.
13. *Lowe D.* Distinctive Image Features from Scale-Invariant Keypoints // International Journal of Computer Vision. — 2004. — Ноябрь. — Т. 60. — С. 91—110. — DOI: 10.1023/B%3AVISI.0000029664.99615.94.
14. An Evaluation of Popular Copy-Move Forgery Detection Approaches / V. Christlein [и др.] // IEEE Transactions on Information Forensics and Security. — 2012. — Дек. — Т. 7. — С. 1841—1854. — DOI: 10.1109/tifs.2012.2218597.
15. *Lowe D.* Object Recognition from Local Scale-Invariant Features // Proceedings of the IEEE International Conference on Computer Vision. — 2001. — Янв. — Т. 2.
16. Geometric tampering estimation by means of a SIFT-based forensic analysis / I. Amerini [и др.] //. — 04.2010. — С. 1702—1705. — DOI: 10.1109/ICASSP.2010.5495485.
17. Copy-Move Forgery Detection: Survey, Challenges and Future Directions / N. Abd Warif [и др.] // Journal of Network and Computer Applications. — 2016. — Сент. — Т. 75. — DOI: 10.1016/j.jnca.2016.09.008.
18. *Bay H., Tuytelaars T., Van Gool L.* SURF: Speeded up robust features // Т. 3951. — 07.2006. — С. 404—417. — DOI: 10.1007/11744023_32.
19. Going deeper into copy-move forgery detection: Exploring image telltales via multi-scale analysis and voting processes / E. Silva [и др.] // JOURNAL OF VISUAL COMMUNICATION AND IMAGE REPRESENTATION. — 2015. — Т. 29. — С. 16.
20. *Kaura W. C. N., Dhavale S.* Analysis of SIFT and SURF features for copy-move image forgery detection // 2017 International Conference on Innovations in Information, Embedded and Communication Systems (ICIIECS). — 2017.

21. *Huang H., Guo W., Zhang Y.* Detection of copy-move forgery in digital images using SIFT algorithm // Т. 2. — IEEE Computer Society. 2008. — С. 272—276.
22. *Bayar B., Stamm M. C.* A Deep Learning Approach to Universal Image Manipulation Detection Using a New Convolutional Layer // — Association for Computing Machinery, 2016. — С. 5—10. — DOI: 10.1145/2909827.2930786.
23. *Zhang R., Isola P., Efros A. A.* Colorful Image Colorization // — Springer International Publishing, 2016. — С. 649—666.
24. *Ronneberger O., Fischer P., Brox T.* U-Net: Convolutional Networks for Biomedical Image Segmentation // — Springer International Publishing, 2015. — С. 234—241.
25. Learning Rich Features for Image Manipulation Detection / P. Zhou [и др.] // — 2018. — С. 1053—1061. — DOI: 10.1109/CVPR.2018.00116.
26. *Fridrich J., Kodovsky J.* Rich Models for Steganalysis of Digital Images // IEEE Transactions on Information Forensics and Security. — 2012. — Т. 7, № 3. — С. 868—882. — DOI: 10.1109/TIFS.2012.2190402.
27. Customized Convolutional Neural Network for Accurate Detection of Deep Fake Images in Video Collections / D. Gura [и др.] // Computers, Materials & Continua. — 2024. — Т. 79. — С. 1—10.
28. *Jing D., Wei W.* CASIA Image Tampering Detection Evaluation Database // — 2013.
29. Welcome to Python.org [Электронный ресурс]. — Режим доступа URL: <https://www.python.org/> (дата обращения: 23.11.2025).
30. Visual Studio Code. — Режим доступа URL: <https://code.visualstudio.com/> (дата обращения: 23.11.2025).
31. Библиотека torch. — Режим доступа URL: <https://pytorch.org/> (дата обращения: 24.10.2025).
32. Библиотека streamlit. — Режим доступа URL: <https://streamlit.io/> (дата обращения: 24.11.2025).

33. Windows 10 «Домашняя для одного языка». — Режим доступа URL: <https://tehnichka.pro/windows-10-home-for-one-language/> (дата обращения: 23.04.2026).
34. Intel Core i5-1135G7. — Режим доступа URL: <https://www.notebookcheck.ru/Intel-Core-i5-1135G7-Testirovanie-processora.507554.0.html> (дата обращения: 23.04.2026).

ПРИЛОЖЕНИЕ А

Листинг А.1 – Реализация структуры сверточной нейронной сети

```
1 class MyCNN(nn.Module):
2     def __init__(self, data_size):
3         super(MyCNN, self).__init__()
4
5         bh = data_size[0]
6         bw = data_size[1]
7
8         bn = 32
9
10        self.conv1 = nn.Sequential(
11            nn.Conv2d(3, bn, kernel_size=3, padding=1),
12            nn.BatchNorm2d(bn),
13            nn.ReLU(True)
14        )
15        self.conv1b = nn.Sequential(
16            nn.Conv2d(bn, bn, kernel_size=4, padding=1,
17            stride=2),
18            nn.BatchNorm2d(bn),
19            nn.ReLU(True)
20        )
21        self.conv1c = nn.Conv2d(3, bn, kernel_size=1,
22            padding=0, stride=1)
23        self.conv1l = nn.Sequential(
24            nn.Conv2d(3, bn, kernel_size=3, padding=1),
25            nn.BatchNorm2d(bn),
26            nn.ReLU(True)
27        )
28        self.conv1lb = nn.Sequential(
29            nn.Conv2d(bn, bn, kernel_size=4, padding=1,
30            stride=2),
31            nn.BatchNorm2d(bn),
32            nn.ReLU(True)
33        )
34        self.conv1lada = nn.Sequential(
35            nn.Conv2d(bn, bn, kernel_size=1, padding=0,
36            stride=1),
37            nn.BatchNorm2d(bn),
38            nn.ReLU(True)
39        )
40        self.conv2 = nn.Sequential(
41            nn.Conv2d(2 * bn, 2 * bn, kernel_size=3,
```

Продолжение листинга A.1

```

1         nn.Conv2d(2 * bn, 2 * bn, kernel_size=4,
2         padding=1, stride=2),
3         nn.BatchNorm2d(2 * bn),
4         nn.ReLU(True)
5     )
6     self.conv2c = nn.Conv2d(3, bn, kernel_size=2,
7     padding=0, stride=2)
8     self.conv22 = nn.Sequential(
9         nn.Conv2d(bn, bn, kernel_size=3, padding=1)
10    ,
11    nn.BatchNorm2d(bn),
12    nn.ReLU(True)
13    )
14    self.conv22b = nn.Sequential(
15        nn.Conv2d(bn, bn, kernel_size=4, padding=1,
16        stride=2),
17        nn.BatchNorm2d(bn),
18        nn.ReLU(True)
19    )
20    self.conv22ada = nn.Sequential(
21        nn.Conv2d(bn, bn, kernel_size=1, padding=0,
22        stride=1),
23        nn.BatchNorm2d(bn),
24        nn.ReLU(True)
25    )
26    self.conv3 = nn.Sequential(
27        nn.Conv2d(3 * bn, 4 * bn, kernel_size=3,
28        padding=1),
29        nn.BatchNorm2d(4 * bn),
30        nn.ReLU(True)
31    )
32    self.conv3b = nn.Sequential(
33        nn.Conv2d(4 * bn, 4 * bn, kernel_size=4,
34        padding=1, stride=2),
35        nn.BatchNorm2d(4 * bn),
36        nn.ReLU(True)
37    )
38    self.conv3c = nn.Conv2d(3, bn, kernel_size=4,
39    padding=0, stride=4)
40    self.conv33 = nn.Sequential(
        nn.Conv2d(bn, bn, kernel_size=3, padding=1)
    ,
        nn.BatchNorm2d(bn),
        nn.ReLU(True)
    )
    self.conv33b = nn.Sequential(
        nn.Conv2d(bn, bn, kernel_size=4, padding=1,
        stride=2),
        nn.BatchNorm2d(bn),
        nn.ReLU(True)
    )

```


Продолжение листинга A.1

```

1         )
2         self.conv33ada = nn.Sequential(
3             nn.Conv2d(bn, bn, kernel_size=1, padding=0,
4                 stride=1),
5             nn.BatchNorm2d(bn),
6             nn.ReLU(True)
7         )
8         self.conv4 = nn.Sequential(
9             nn.Conv2d(5 * bn, 8 * bn, kernel_size=3,
10                 padding=1),
11             nn.BatchNorm2d(8 * bn),
12             nn.ReLU(True)
13         )
14         self.conv4b = nn.Sequential(
15             nn.Conv2d(8 * bn, 8 * bn, kernel_size=4,
16                 padding=1, stride=2),
17             nn.BatchNorm2d(8 * bn),
18             nn.ReLU(True)
19         )
20         self.conv4c = nn.Conv2d(3, bn, kernel_size=8,
21             padding=0, stride=8)
22         self.conv44 = nn.Sequential(
23             nn.Conv2d(bn, bn, kernel_size=3, padding=1)
24             ,
25             nn.BatchNorm2d(bn),
26             nn.ReLU(True)
27         )
28         self.conv44b = nn.Sequential(
29             nn.Conv2d(bn, bn, kernel_size=4, padding=1,
30                 stride=2),
31             nn.BatchNorm2d(bn),
32             nn.ReLU(True)
33         )
34         self.conv44ada = nn.Sequential(
35             nn.Conv2d(bn, bn, kernel_size=1, padding=0,
36                 stride=1),
37             nn.BatchNorm2d(bn),
38             nn.ReLU(True)
39         )
40         self.conv5 = nn.Sequential(
41             nn.Conv2d(9 * bn, 8 * bn, kernel_size=3,

```

Продолжение листинга A.1

```

1         nn.ReLU(True)
2     )
3     self.conv55ada = nn.Sequential(
4         nn.Conv2d(bn, bn, kernel_size=1, padding=0,
5         stride=1),
6         nn.BatchNorm2d(bn),
7         nn.ReLU(True)
8     )
9     self.deconv4 = nn.Sequential(
10        nn.Upsample(size=(bw, bh), mode='bilinear')
11    ,
12        nn.Conv2d(9 * bn, 8 * bn, kernel_size=1),
13    )
14    self.att4 = SELayer(channel=10*bn, reduction
15    =16)
16    self.deconv4b = nn.Sequential(
17        nn.Conv2d(10 * bn, 8 * bn, kernel_size=3,
18        padding=1),
19        nn.BatchNorm2d(8 * bn),
20        nn.ReLU(True)
21    )
22    self.deconv44 = nn.Sequential(
23        nn.Upsample(size=(bw, bh), mode='bilinear')
24    ,
25        nn.Conv2d(bn, bn, kernel_size=1),
26    )
27    self.deconv44b = nn.Sequential(
28        nn.Conv2d(bn, bn, kernel_size=3, padding=1)
29    ,
30        nn.BatchNorm2d(bn),
31        nn.ReLU(True)
32    )
33    self.deconv44ada = nn.Sequential(
34        nn.Conv2d(bn, bn, kernel_size=1, padding=0,
35        stride=1),
36        nn.BatchNorm2d(bn),
37        nn.ReLU(True)
38    )
39    self.deconv3 = nn.Sequential(
40        nn.Upsample(size=(2 * bw, 2 * bh), mode='
41        bilinear'),
42        nn.Conv2d(8 * bn, 4 * bn, kernel_size=1),
43    )
44    self.att3 = SELayer(channel=6 * bn,
45    reduction=16)
46    self.deconv3b = nn.Sequential(
47        nn.Conv2d(6 * bn, 4 * bn, kernel_size

```

Продолжение листинга A.1

```

1         =3, padding=1),
2             nn.BatchNorm2d(4 * bn),
3             nn.ReLU(True)
4         )
5         self.deconv33 = nn.Sequential(
6             nn.Upsample(size=(2 * bw, 2 * bh),
mode='bilinear'),
7             nn.Conv2d(bn, bn, kernel_size=1),
8         )
9         self.deconv33b = nn.Sequential(
10             nn.Conv2d(bn, bn, kernel_size=3, padding=1)
11         ,
12             nn.BatchNorm2d(bn),
13             nn.ReLU(True)
14         )
15         self.deconv33ada = nn.Sequential(
16             nn.Conv2d(bn, bn, kernel_size=1, padding=0,
stride=1),
17             nn.BatchNorm2d(bn),
18             nn.ReLU(True)
19         )
20         self.deconv2 = nn.Sequential(
21             nn.Upsample(size=(4 * bw, 4 * bh), mode='
bilinear'),
22             nn.Conv2d(4 * bn, 2 * bn, kernel_size=1),
23         )
24         self.att2 = SELayer(channel=4 * bn, reduction
=16)
25         self.deconv2b = nn.Sequential(
26             nn.Conv2d(4 * bn, 2 * bn, kernel_size=3,
padding=1),
27             nn.BatchNorm2d(2 * bn),
28             nn.ReLU(True)
29         )
30         self.deconv22 = nn.Sequential(
31             nn.Upsample(size=(4 * bw, 4 * bh), mode='
bilinear'),
32             nn.Conv2d(bn, bn, kernel_size=1),
33         )
34         self.deconv22b = nn.Sequential(
35             nn.Conv2d(bn, bn, kernel_size=3, padding=1)
36         ,
37             nn.BatchNorm2d(bn),
38             nn.ReLU(True)
39         )
40         self.deconv22ada = nn.Sequential(
41             nn.Conv2d(bn, bn, kernel_size=1, padding=0,
stride=1),
42             nn.BatchNorm2d(bn),
43             nn.ReLU(True)

```

Продолжение листинга A.1

```

1         )
2         self.deconv1 = nn.Sequential(
3             nn.Upsample(size=(8 * bw, 8 * bh), mode='
bilinear'),
4             nn.Conv2d(2 * bn, bn, kernel_size=1),
5         )
6         self.att1 = SELayer(channel=3 * bn, reduction
=16)
7         self.deconv1b = nn.Sequential(
8             nn.Conv2d(3 * bn, bn, kernel_size=3,
padding=1),
9             nn.BatchNorm2d(bn),
10            nn.ReLU(True)
11        )
12        self.deconv1l = nn.Sequential(
13            nn.Upsample(size=(8 * bw, 8 * bh), mode='
bilinear'),
14            nn.Conv2d(bn, bn, kernel_size=1),
15        )
16        self.deconv1lb = nn.Sequential(
17            nn.Conv2d(bn, bn, kernel_size=3, padding=1)
18        ,
19            nn.BatchNorm2d(bn),
20            nn.ReLU(True)
21        )
22        self.deconv1lada = nn.Sequential(
23            nn.Conv2d(bn, bn, kernel_size=1, padding=0,
stride=1),
24            nn.BatchNorm2d(bn),
25            nn.ReLU(True)
26        )
27        self.conv_end = nn.Conv2d(bn, 2, kernel_size=3,
padding=1, bias=False)
28        self.conv_end_label_edge = nn.Conv2d(bn, 2,
kernel_size=3, padding=1, bias=False)
29        self.conv_end_img_edge = nn.Conv2d(bn, 1,
kernel_size=3, padding=1, bias=False)
30
31        def forward(self, data, label=None):
32            L1 = self.conv1(data)
33            L1b = self.conv1b(L1)
34            L1c = self.conv1c(data)
35            L1l = self.conv1l(data)
36            L1lb = self.conv1lb(L1l)
37            L1la = self.conv1lada(L1lb)
38            L1lada = L1la + L1lb
39            L1b2 = torch.cat([L1b, L1lada], 1)
40
41            L2 = self.conv2(L1b2)

```

Продолжение листинга А.1

```

1      L2b = self.conv2b(L2)
2      L2c = self.conv2c(data)
3      L22 = self.conv22(L11b)
4      L22b = self.conv22b(L22)
5      L22a = self.conv22ada(L22b)
6      L22ada = L22a+L22b
7      L2b2 = torch.cat([L2b, L22ada], 1)
8
9      L3 = self.conv3(L2b2)
10     L3b = self.conv3b(L3)
11     L3c = self.conv3c(data)
12     L33 = self.conv33(L22b)
13     L33b = self.conv33b(L33)
14     L33a = self.conv33ada(L33b)
15     L33ada = L33a + L33b
16     L3b2 = torch.cat([L3b, L33ada], 1)
17
18     L4 = self.conv4(L3b2)
19     L4b = self.conv4b(L4)
20     L4c = self.conv4c(data)
21     L44 = self.conv44(L33b)
22     L44b = self.conv44b(L44)
23     L44a = self.conv44ada(L44b)
24     L44ada = L44a + L44b
25     L4b2 = torch.cat([L4b, L44a], 1)
26
27     L5 = self.conv5(L4b2)
28     L55 = self.conv55(L44b)
29     L55a = self.conv55ada(L55)
30     L55ada = L55a + L55
31     L52 = torch.cat([L5, L55ada], 1)
32
33     DL4 = self.deconv4(L52)
34     DL4_add = DL4 + L4
35     DL44 = self.deconv44(L55)
36     DL44_add = DL44 + L44
37     DL44b = self.deconv44b(DL44_add)
38     DL44bada = DL44b + DL44_add
39     DL4_cat = torch.cat([DL4_add, L4c, DL44bada],
1)
40     DL4_att = self.att4(DL4_cat)
41     DL4b = self.deconv4b(DL4_att)
42
43     DL3 = self.deconv3(DL4b)
44     DL3_add = DL3 + L3
45     DL33 = self.deconv33(DL44b)
46     DL33_add = DL33 + L33
47     DL33b = self.deconv33b(DL33_add)
48     DL33ba = self.deconv33ada(DL33b)
49     DL33bada = DL33ba + DL33b

```

Продолжение листинга A.1

```

1      DL3_cat = torch.cat([DL3_add, L3c, DL33bada],
2      1)
3      DL3_att = self.att3(DL3_cat)
4      DL3b_ = self.deconv3b(DL3_att)
5
6      DL2 = self.deconv2(DL3b)
7      DL2_add = DL2 + L2
8      DL22_ = self.deconv22(DL33b)
9      DL22_add = DL22 + L22
10     DL22b_ = self.deconv22b(DL22_add)
11     DL22ba = self.deconv22ada(DL22b_)
12     DL22bada = DL22ba + DL22_add
13     DL2_cat = torch.cat([DL2_add, L2c, DL22bada],
14     1)
15     DL2_att = self.att2(DL2_cat)
16     DL2b_ = self.deconv2b(DL2_att)
17
18     DL1 = self.deconv1(DL2b)
19     DL1_add = DL1 + L1
20     DL11_ = self.deconv11(DL22b)
21     DL11_add = DL11 + L11
22     DL11b_ = self.deconv11b(DL11_add)
23     DL11ba = self.deconv11ada(DL11b_)
24     DL11bada = DL11ba + DL11b
25     DL1_cat = torch.cat([DL1_add, L1c, DL11bada],
26     1)
27     DL1_att = self.att1(DL1_cat)
28     DL1b_ = self.deconv1b(DL1_att)
29
30     end = self.conv_end(DL1b)
31     end_label_edge = self.conv_end_label_edge(DL1b)
32     end_img_edge = self.conv_end_img_edge(DL11b)
33
34     output = torch.argmax(end, dim=1)
35     output2 = torch.argmax(end_label_edge, dim=1)
36     output3 = end_img_edge
37     if not self.training:
38         return output
39     return end, end_label_edge, end_img_edge,
40     output, output2, output3

```

ПРИЛОЖЕНИЕ Б

Презентация к выпускной квалификационной работе состоит из 16 слайдов.